Python Fundamentals (Part1)

Concepts : Output/ Input, Variables, Data Types & Operators

Output

The first & the easiest thing that we can probably do in Python is printing (or outputting) something on our screen and we can perform this simple task using a special function, called the `print()` function.
What exactly is a function? We'll learn about it in the next chapter but for now we can imagine it to be a specialized helper that performs a specific task. So, `print()` is our little helper that helps us display something on the screen(the console). We can use it to show information, results of calculations or even format our text.

Syntax:

```
print("hello world")
# output will be 'Hello World'
```

Anything quotes('') or double-quotes("") is called a string & gets printed as it is.

We can also print numerical values:

```
print(3.14)      # output: 3.14
print(2 + 3)     # output: 5
```

Each print statement displays output on different lines but we can also display multiple values on same line.

```
print("hello world", "from PRIME")
```

Python Character Set

A Character set is a collection of characters, numbers, symbols that our language recognizes & we can use. Following are the common characters we'll be frequently using in Python:

1. English letters - `'A'` to `'z'` and `'a'` to `'z'`
2. Digits - `0` to `9`
3. Special symbols - `+`, `-`, `*`, `/`, `%`, etc.
4. Whitespaces & Escape characters - blank space ( ), tab space (`\t`), newline (`\n`) etc.
5. All ASCII & Unicode characters as part of data & literals

Variables

A **variable** is like a **container** that stores **data** or **values** in our program. We can think of it as a labeled box where we can place something in, change it, or use it later.
There are variables in Python:
1. Have names called identifiers.
2. Can store data of any type (numbers, text etc.)

Some **examples**:

```
name = "Shradha"
age = 30
PI = 3.14
word = "arificial intelligence"

print ("value of PI =", PI)
```

Variable Naming(identifier) Rules

1. Name must start with a **letter** (a–z, A–Z) or **underscore** `_`.
2. Can contain **letters, numbers, or underscores** after the first character.
3. Cannot use **Python keywords** (like `if`, `for`, `class`) as variable names.

Let's look at some invalid variable names:

```
2name = "Bob"    # Starts with a number
class = 10       # 'class' is a keyword
```

Important Note

1. Python is **dynamically typed**. So when creating variables we don't need to declare type explicitly.
2. Python is a **case-sensitive** i.e. variable `age` & `Age` are different.
3. **Indentation** in Python is important & the general standard is 4 spaces per indentation level.
4. **Keywords** are reserved words in Python & their meanings are predefined by the language.
   Examples - `if`, `for`, `class`, `while`, `else`, `in`, `def` etc.

Data Types

A **data type** defines the **kind of value a variable can hold**. Python has several built-in data types.
Let's have a look at the simplest ones to start with:

1. `int` - integer values (whole numbers)
2. `float` - floating-point values (decimal numbers)
3. `str` - strings (sequence of characters like words & sentences)
4. `bool` - boolean values (`True` / `False`)
5. `None` - special constant representing no value / null

We also have a lot of other data types like `complex` , `list` , `tuple` , `set` etc. that we'll cover in coming chapters. To check the variable type we can use the `type()` function.

```
x = 10
print(type(x))          # <class 'int'>

PI = 3.14
print(type(PI))         # <class 'float'>

name = "shradha"
print(type(name))       # <class 'str'>

isTeacher = True
print(type(isAdult))    # <class 'bool'>

empty_var = None
print(type(empty_var)) # <class 'NoneType'>
```

Type Conversion & Type Casting

Type conversion is when we convert(cast) variables from one type to another. It can happen in 2 ways:

1. Type conversion - Implicit, done automatically by Python

```
a = 5
b = 3.0
print(a + b) # Python converts ans in float by default
```

2. **Type Casting** - Explicitly, done by the programmer

```
x = 10          # int
y = float(x)    # convert to float
z = str(x)      # convert to string
```

`int()`, `float()`, `str()`, `bool()`, `list()`, `tuple()` are common type conversion functions.

Operators

An **operator** is a symbol that tells Python to perform a **specific computation** on one or more values (called operands).
Example:

```
print(1 + 3) # '+' is an Operator & (1, 3) are operands
```

There are several types of operators in Python. Let's start by understanding some of the most common ones:

1. **Arithmetic Operators** - used to perform math operations.

   Operators - `+`, `-`, `*`, `/`, `%`, `**`

```
a = 5
b = 10

print(a + b)     # Addition
print(a - b)     # Subtraction
print(a * b)     # Multiplication
print(a / b)     # Division
print(a % b)     # Modulo - remainder
print(a ** b)    # Power
```

2. **Relational Operators** - used to compare values.

Operators - `==` , `!=` , `>` , `>=` , `<` , `<=`

```
a = 10
b = 20

# Equal to
print("a == b:", a == b) # False

# Not equal to
print("a != b:", a != b) # True

# Greater than
print("a > b:", a > b) # False

# Less than
print("a < b:", a < b) # True

# Greater than or equal to
print("a >= b:", a >= b) # False

# Less than or equal to
print ("a <= b:", a <= b) # True
```

3. **Assignment Operators** - used to assign values to variables.

Operators - `=` , `+=` , `-=` , `*=` , `/=` , `%=` , `**`

```
# Simple assignment
x = 10
print("x =", x) # 10

x += 5 # equivalent to x = x + 5
print("x += 5 ->", x) # 15

x -= 3 # equivalent to x = x - 3
print("x -= 3 ->", x) # 12

x *= 2 # equivalent to x = x * 2
print("x *= 2 ->", x) # 24

x /= 4 # equivalent to x = x / 4
print("x /= 4 ->", x) # 6.0

x %= 4 # equivalent to x = x % 4
print("x %= 4 ->", x) # 2.0

x **= 3 # equivalent to x = x ** 3
print("x **= 3 ->", x) # 8.0
```

4. Logical Operators - used to combine boolean values.

Operators - `and` , `or` , `not`

val1	val2	and
T	T	T
T	F	F
F	T	F
F	F	F

val1	val2	or
T	T	T
T	F	T
F	T	T
F	F	F

val1	not
T	F
F	T

```
x = 10
y = 20
z = 5

# AND
print(x > z and y > x) # True

# OR
print(x > y or y > z) # True

# NOT
print(not (x > y)) # True
```

We'll cover more operator types like membership operators in future chapters.

Operator Precendence

Operators have a **priority**; it means there is a specific order in which operations are performed when we have multiple operators appearing in the same expression. The order is as follows:

- `()` - Parentheses (highest priority)
- `**` - Exponent
- `+x, -x, ~x` - Unary operators
- `/ // %` - Multiplication, division, floor, modulus
- `+ -` - Addition, subtraction
- `<< >>` - Bitwise shifts
- `&` - Bitwise AND
- `^` - Bitwise XOR
- `|` - Bitwise OR
- Comparison operators - `<, <=, >, >=, !=, ==`
- `not`
- `and`
- `or`
- Assignment operators - `=, +=, -= ...`

Note - Unary operators are operators that **works on a single operand** (a single value or variable) to perform an operation. These are like `not`, `+` (unary plus), `-` (unary minus) etc.
`+x` is used to indicate positive value & `-x` to indicate negative value.

```
x = 5
y = -x # unary minus
print(x) # 5
print(y) # -5
```

We don't generally use +x as numbers are positive by default.

Input

We use the `input()` function to take input from user while the program is running. Whatever the user types in input prompt is returned as a string.

Syntax:

```
name = input("enter name: ")
print(name)
print(type(name))    # type is string
```

We can use type conversion functions to convert the type of input.

```
# program to add 2 numbers entered by the user
a = int(input("enter 1st num: "))
b = int(input("enter 2nd num: "))

print(a + b)
```

Let's summarize all the concepts we learnt in this chapter into a simple practice problem.
Write a program that takes input of 2 numbers (`a` & `b`) and print the average.

```
a = int(input("enter 1st num: ")) # 5
b = int(input("enter 2nd num: ")) # 10

avg = (a + b) / 2
print("average = ", avg) # 7.5
```

Keep Learning & Keep Exploring!

Python Fundamentals (Part2)

Concepts : Conditional Statements, Loops & Functions

Conditional Statements

Conditional statements let our program **decide what to do based on conditions** (i.e. true or false expressions).

There are 3 main conditional statements in Python:

1. `if` - used to test a condition. If it's **True**, the indented block runs.
2. `else` - else block runs if all above conditions are false.
3. `elif` - ("else if") used when we have **multiple conditions** to check.

Let's practice conditional statements with the help of following examples.

```
# Example 1
age = int(input("enter age: "))

if age >= 18:            # if true/false, entire block of code is executed
    print("you can vote")
    print("you can drive")

else:
    print("you can't vote")
    print("you can't drive")
```

```
# Example 2 - Traffic Lights
color = input("enter color: ")

if color == "red":
    print("Stop")
elif color == "yellow":
    print("Look")
elif color == "green":
    print("Go")
else:
    print("wrong color")
```

Note - We can have standalone & multiple `if` statements but `elif` & `else` can't be used without an `if` statement preceding them.

Apart from simple conditions we can also check for multiple conditions using Logical operators like `and` , `or` and `not` in expressions.

```
# Example 3 - Logical operators in expressions
age = int(input("enter age: "))

if age < 13:
    print("child")
elif (age >= 13 and age < 18):
    print("teenager")
else:
    print("adult")
```

Nesting in Conditionals

Nesting means **placing one block of code inside another**. In the context of conditional statements, it means putting one `if` (or `if-elif-else`) inside another. This helps when you need to make a decision based on another decision.

```
# Login System

username = input("enter username: ")
password = input("enter password: ")

if (username == "admin" and password == "pass"):
   print("log in successful!")
else:
 if username != "admin":          # NESTING
        print("wrong user name, try again.")
    else:
        print("wrong password, try again.")
```

A login system that checks for correct/ incorrect credentials

Ternary Statements

Ternary statements (or conditional expressions) are just another compact way to writing our `if-else` statements in one-line.

Ternary statement **Syntax**:

```
value_if_true if condition else value_if_false
```

One line decided between 2 values based on condition.

Let's see an example:

```
age = 18
status = "Adult" if age >= 18 else "Not Adult"
print(status)
```

> Note - Everything that we write with ternary statements can be done with normal if-else statements. Ternary statements are generally only used for simple conditions, not recommended for nested or complex conditions.

Match Case Statements

In Python, the `match` / `case` statements are an alternative to long chain of `If..elif..else` statements. They were introduced in Python 3.10 & we generally don't see them much in production code.

Match case **Syntax**:

```
match variable:
   case pattern1:
        # code to run if pattern1 matches
   case pattern2:
       # code to run if pattern2 matches
   case _:
       # default case (like 'else')
```

1. `match` - what you're comparing
2. `case` - the value or pattern to match against
3. `_` - wildcard (matches anything, like "default")

Let's see an example:

```
color = input("enter color: ")

match color:
    case "red":
       print("Stop")
    case "yellow":
       print("Look")
    case "green":
       print("Go")
    case _:
      print("wrong color")
```

Simple value match for traffic lights

Practice Problems (Set 1)

1. For a given number `n`, print if it's a multiple of 5 or not.
2. For a given number `n`, print if it's Odd or Even.

Loops

A **loop** lets us **execute a block of code multiple times:** either a set number of times or until a condition is met.

Python has 2 main types of loops:

1. `while` loop - repeat while a condition is `True`
2. `for` loop - iterate over a sequence (like a list, string, or range)

The while Loops

Syntax of a `while` loop:

```
while condition:
    # code block
```

Whatever statements we write in the code block are all executed one after the other

Let's look at some examples:

```
# Example 1 - print 1 to 5
i = 1
while I <= 5:
    print(i)
    i += 1

# Example 2 - print 5 to 1
i = 5
while i > 0:
    print(i)
    i -= 1
```

If the loop expression is such that it always computes `True`, then such a loop never stops & is called an **infinite loop**. We should try to never unintentionally create such a loop.

```
while True: # DO NOT RUN this code
    print("Prime")
```

Example of an Infinite Loop

Loop control statements

Python gives us some tools to **control** how loops behave & a classical example of them are 2 statement - `break` & `continue`.

The `break` keyword

It stops the loop immediately.

```
# Break for multiple of 6
i = 1

while i <= 10:
    if(i % 6 == 0):
        break
    print(i)
    i += 1

# output: 1, 2, 3, 4, 5
```

The `continue` keyword

It skips the rest of the current iteration and moves to the next one.

```
# Skip multiples of 3
i = 0

while(i < 10):
    i += 1
    if(i % 3 == 0):
        continue;
    print(i)

# output: 1, 2, 4, 5, 7, 8, 10
```

The for Loops

In Python, a `for` loop is used to iterate (loop) over a sequence, such as a list, tuple, string, or range, and execute a block of code for each item in that sequence.

Syntax of a `for` loop:

```
for variable in sequence:
    # code to run for each item
```

Let's look at an example:

```
for i in range(5):
    print(i)

# output: 0, 1, 2, 3, 4
```

The `in` keyword that we used is a **membership operator** in Python, used to loop over a sequence or to check the presence of an item in a sequence, like a string.

```
word = "Prime"

# Example 1 - Looping over a string
for ch in word:
    print(ch)

# Example 2 - Check if char 'i' exists in word
if 'i' in word:
    print("letter exists")

# Example 3 - Count number of i's in the word
word = "artificial intelligence"
count = 0

for ch in word:
    if ch == 'i':
        count += 1

print(f"i occurs {count} times.")
```

Just like conditionals, we can also have **nested loops** i.e. loop inside a

```
for i in range(1, 3):
    for j in range(1, 3):
    print(f"({i}, {j})")

# output: (1, 2) (1, 2) (2, 1) (2, 2)
```

Nested Loop Example

The `range()` Function

The `range()` function in Python is used to generate a **sequence of numbers**, typically for use in loops. It doesn't actually create a list in memory (unless we convert it); instead, it creates an iterator that produces numbers one by one, which makes it very efficient.

The function has 3 parameters:

1. start (default=0) - the number to start from
2. stop - the number to stop before (not included)
3. step (default=1) - how much to increase by each time

```
# single argument - start
for i in range(5):
    print(i)

# output: 0, 1, 2, 3, 4

# 2 arguments - start, stop
for i in range(1, 6):
    print(i)

# output: 1, 2, 3, 4 , 5

# 3 arguments - start, stop, step
for i in range(1, 10, 2):
    print(i)

# output: 1, 3, 5, 7, 9
```

Practice Problems (Set 2)

1. Print multiplication table for any number `n` . [using `while`]
2. Print odd numbers from 1 to 10, using continue. [using `while`]
3. Count vowels in a word. [using `for`]
4. Sum of first `n` natural numbers. [using `for`]

Functions

Functions in Python are **blocks of reusable code** that perform a specific task. They make our code organized, readable, and easier to maintain.

Function Definition

We use the `def` keyword to define our function:

```
# Function Definition
def hello():
    print("hello from Prime")
```

and to call the function i.e. to run it we write their name with parentheses:

```
hello() # Function Call
```

Function call actually invokes the function & then all the block of code written in the functions definition is executed.

Functions with Parameters

We can also pass data to a function using **parameters** (inside the parentheses). Values actually passed when calling the function are called **arguments**.

```
# Fnx to compute sum of 2 nums

def sum(a, b):          # a & b are parameters
    print (a + b)

print(sum(5, 10))       # 5 & 10 are arguments
```

wherever fnx → it means function

Functions can **return** a result using the `return` keyword.

```
# Fnx to computer average of 3 nums

def avg(a, b, c):
    return (a + b + c) / 3
print(avg(1, 2, 3))
```

Default Parameters

We can assign **default values** to function parameters so that if the caller doesn't provide that argument, Python uses the default values. And these default parameters must come last. We cannot have a non-default argument after a default argument.

```
# sum() fnx with default param 1
def sum(a, b = 1): # default val of b is 1
    return a + b

print(sum(5)) # output: 6
```

Built-in Functions

All the functions that we have studied till now are **user-defined functions**, but in Python we also have a lot of built-in functions too. The definition (logic) of **built-in functions** is already written in Python & we just have to use them.
Some built-in functions that we have already used till now:

1. `print()`
2. `input()`
3. `type()`
4. `range()`

Lambda Functions

Lambda functions are short, one-liner functions that are used to perform simple tasks. These are created using the `lambda` keyword. These functions are anonymous & do not have a name like regular functions defined with `def` .
Lambda function **Syntax**:

```
lambda arguments: expression
# arguments -> 1 or more parameters
# express ion -> single expression whose result is automatically returned
```

Let's see an example of a lambda function that computes square of a number:

```
# fnx to compute x^2
square = lambda x: x * x
print(square(5))
```

Practice Problems (Set 3)

1. Create a function to compute factorial of a number `n` .

2. Create a function to return largest of 3 numbers - `a` , `b` & `c` .

Solutions (Set 1 - Conditionals)

1. Multiple of 5 or not

```
num = int(input("enter number: "))

if num % 5 == 0:
    print("multiple of 5")
else:
    print("NOT a multiple of 5")
```

2. Odd or Even

```
num = int(input("enter number: "))

if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Solutions (Set 2 - Loops)

1. Multiplication table of `n`

```
n = int(input("enter n: "))
i = 1

while i <= 10:
      print(i * n)
      i += 1
```

2. Odd nums from 1 to 10, using continue

```
i = 0

while(i < 10):
       i += 1
       if(i % 2 == 0):
              continue;
       print(i)

# output: 1, 3, 5, 7, 9
```

3. Count vowels

```
for ch in word:
    if (ch   == 'a' or ch == 'a' or ch == 'a' or ch == 'a' or ch == 'a'):
        count += 1

print(f"vowel count = {count}")
```

4. Sum of first n Natural numbers

```
n = int(input("enter n: "))
sum = 0
for i in range(1, n+1):
    sum += i

print("sum = ", sum)
```

Solutions (Set 3 - Functions)

1. Factorial of N

```python
# Factorial of N
n = int(input("enter n: "))

fact = 1
for i in range (1, n+1):
fact *= i

print("factorial = ", fact)
```

2. Largest of 3 numbers

```python
def get_largest (a, b, c):
    if (a > b and a > c):
        return a
    elif b > c:
        return b
    else:
        return c
print (get_largest (3, 10, 5))
```

> *Keep Learning & Keep Exploring!*

Python Fundamentals (Part3)

Concepts : String, List, Tuple, Dictionary & Set

In this chapter we are going to dive deep into some of the not-so-simple data types of Python.

Strings

What is a String?

A **string** in Python is a sequence of characters enclosed in quotes:

```
str1 = "hello world"
str2 = 'Prime'
```

Strings are **immutable**, meaning once created, their contents can't be changed directly.

`len()` Function

We can use the built-in `len()` function to print the length of a string:

```
word = 'Prime'
print(len(word))        # 5
```

Concatenation

Just like we add numbers, we can concatenate (i.e. join) strings using the `+` operator.

```
str1 = "Apna"
str2 = "College"

word = str1 + " " + str2        # concatenation
print(word)
```

Loop on Strings

```
s = "Python"
for ch in s:     # ch will store individual chars - 'P', 'y', 't' & so on
    print(ch)
```

Indexing in Strings

Index in a sequence (like strings) represents the position value of individual elements i.e. characters in case of a string. So Indexing lets us access individual characters in a string.

Python follows **Zero-based indexing** i.e. first character is at index `0`.

```
s = "Python"
print(s[0])  # 'P'
print(s[3])  # 'h'
print(s[-1]) # 'n' (negative index: last character)
```

Slicing in Strings

Slicing is a powerful feature of Python that lets us access multiple elements at once. We can do slicing in strings & even on other sequences like lists & tuples.
The general syntax for slicing a string is:

```
string[start : end : step]
```
(stop)

Where:

- **start** - index where the slice starts (inclusive). Defaults to `0` if omitted.
- **stop** - index where the slice ends (exclusive). Defaults to the end of the string if omitted.
- **step** - how many indices to move forward each time. Defaults to `1`.

Example:

```
s = "Python"
print(s[0:2])   # 'Py'
print(s[2:])    # 'thon'
print(s[:3])    # 'Py t'
print(s[::2])   # 'Pto' (every second char)
print(s[::-1])  # 'nohtyP' (reversed string)
```

String Formatting

String formatting is the process of creating **dynamic strings** by inserting values from variables or expressions into a predefined string template. This allows for the construction of flexible and informative output based on changing data.
We have multiple ways to format a string, the most modern 2 are:

1. using the `format()` function
2. using `f-strings`

1. Using `.format()`

In this way we use `{}` as placeholders & pass placeholder replacement values in the format function.

```
name = "Rahul"
age = 25

text = "My name is {} and I am {} years old".format(name, age)
print(text)
```

We can also use positional & named placeholders:

```
"Coordinates: {1}, {0}".format("x", "y") # 'Coordinates: y, x'

"Name: {n}, Age: {a}".format(n="Bob", a=30)
```

2. **Using f-strings** (Python 3.6+)
F-strings are concise, readable & will be our preferred way going forward.
In f-strings we prefix the string with `f` and put our variable or expressions inside `{}` .

```
name = "Rahul"
age = 25
text = f"My name is {name} and I am {age} years old"
print(text)
```

You can put any valid Python expression inside `{}` :

```
a = 5
b = 10
print(f"sum of {a} & {b} = {a + b}")
print(f"avg of {a} & {b} = {(a + b) / 2}")
```

Lists

What is a List?

- A **list** is a **collection of items** in Python.
- Items in a list are **ordered, changeable (mutable), and can contain duplicates**.
- Lists can hold any data type: numbers, strings, other lists, etc.
- Lists are written using square brackets `[]`

Example:

```
my_list = [1, 2, 3, 4, 5]
print(my_list)
print(type(my_list))     # <class 'list'>

my_list2 = [10, "Hello", 3.14, True, 10]     # heterogenous list
print(my_list2)
```

List Characteristics

1. **Ordered** – Items have a defined order and can be accessed by index.
2. **Mutable** – Items can be changed after the list is created.
3. **Allows duplicates** – Same value can appear more than once.
4. **Heterogeneous** – Can contain different data types.

List Indexing

Index in list is the position value of an item. Index starts from `0`.
We can use index to access elements, modify the list or even slice it.

Access Elements

```
my_list = ["apple", "banana", "cherry"]
print(my_list[0])  # apple
print(my_list[1])  # banana
print(my_list[-1]) # cherry (last element)
```

Modify Elements

```
my_list = [1, 2, 3, 4]
my_list[0] = 10
print(my_list) # [10, 2, 3, 4]
```

Slicing - Slicing in lists is same as slicing in strings.
The general syntax for slicing a list is:

```
list[start:stop:step]
```
(stop → end)

- **start** - inclusive
- **end** - exclusive
- **step** - optional (default = 1)

```python
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Simple Slice
print(numbers[2:5])  # Output: [2, 3, 4]

print(numbers[:4])   # Output: [0, 1, 2, 3] (from start to index 3)
print(numbers[5:])   # Output: [5, 6, 7, 8, 9] (from index 5 to end)
print(numbers[:])    # Output: [0,1,2,3,4,5,6,7,8,9] (copy of the whole list)

# using STEP
print(numbers[::2])  # Output: [0, 2, 4, 6, 8] (every2nd element)
print(numbers[1::3]) # Output: [1, 4, 7] (start at 1, every 3rd element)

# NEGATIVE slice
print(numbers[-5:-2]) # Output: [5, 6, 7] (negative indexing from end)
```

List Methods

We have a lot of useful functions that are associated with lists. Let's have a look at some of them:

1. `len()` - returns the number of elements in a list i.e. length of the list.
2. `append(element)` - adds an elements to the end.
3. `insert(element, idx)` - adds at a specific position.
4. `sort()` - sorts in ascending/alphabetical order.
5. `reverse()` - flips the order of elements.

```python
nums = [5, 2, 9]
print(len(nums)) # 3

nums.append(7)
print(nums) # [5, 2, 9, 7]

nums.insert(1, 4)
print(nums) # [5, 4, 2, 9, 7]

nums.sort()
print(nums) # [2, 4, 5, 7, 9]

nums.reverse()
print(nums) # [9, 7, 5, 4, 2]
```

There are many more methods associated with lists. It doesn't make a lot of sense to cover all of them theoretically here, so we'll be covering them whenever we use them in coming chapters.

Loops on Lists

We use the classical `for` loop to traverse the entire list element-wise.

The most common way is using a loop in lists is to print elements.

```
numbers = [10, 20, 30, 40, 50]

for num in numbers:
    print(num)
```

Linear Search

A linear search checks each element one by one to find a target (x).

```
numbers = [5, 12, 7, 3, 18, 9]
x = 18
idx = 0

for num in numbers:
    if num == target:
        print(f"{x} found at index={idx}")
        break
~~idx += 1~~
    idx += 1
```

In the above code, we are assuming that `x` always exists in the list.

Tuples

What is a Tuple?

A **tuple** in Python is an ordered, immutable collection of items.
Example:

```
tup = (10, 20, 30)

print(tup)
print(type(tup)) # < class 'tuple'>

empty_tu ple = ()   # empty_tuple

single_element_tuple = (42,)
```

Tuple Characteristics

1. Ordered
2. **Immutable** – Items cannot be changed after the tuple is created.

3. Allows duplicates
4. Heterogeneous

Tuples are very similar to lists. Some of the differences are that tuples are immutable & because of this immutability they are also faster.

Indexing & Slicing (Same as Lists)

```
t = (10, 20, 30, 40)

print(t[0])   # 10
print(t[-1])  # 40
print(t[1:3]) # (20, 30)
```

Loops on Tuples (Same as Lists)

```
t = (10, 20, 30, 40)

for val in t:
    print(val)
```

Using loops to calculate **sum** of all elements in the tuple:

```
t = (5, 15, 25)

sum = 0
for val in t:
    sum val

print("Sum:", sum)
```

Tuple Methods

Let's have a look at some of tuple methods:

1. `index(val)` - returns index of first occurrence for any val
2. `count(val)` - returns total count of occurrence for any val

```
t = (1, 2, 2, 3, 5)

print(t.index(2)) # 1
print(t.count(2)) # 3
```

Dictionaries

What is a Dictionary?

A dictionary is an unordered, mutable collection of **key-value pairs**.
Example:

```
my_dict = {
    "name": "Shradha",
    "age": 30,
    "city": "Delhi"
}
```

Dictionary Characteristics

1. Dictionary Keys must be **unique**
2. Dictionary Keys must be **immutable** (e.g., strings, numbers, tuples)
3. Dictionary is mutable.
4. Dictionary values can be anything (lists, other dictionaries, etc.)
5. Unordered (although in modern Python, dictionaries preserve insertion order)

Accessing values (using key & `[]`)

```
student = {"name": "Bob", "age": 20}
print(student["name"])  # Bob
```

Dictionary Methods

Let's have a look at some of the important dictionary methods:

1. `keys()` - returns all keys
2. `values()` - returns all values
3. `items()` - returns key–value pairs as tuples
4. `get(key)` - a safer way to access value of a particular key. Instead of throwing an error it returns `None` if key doesn't exist
5. `update(new_item)` - adds a new item to the dictionary

```
d = {
    "name": "Shradha,
    "subjects": ["math", "science", "physics"],
    "cgpa": 9.5
}

print(d.keys()) # dict_keys
print(d.values()) # dict_values
print(d.items()) # dict_items

print(d.get("cgpa2")) # return None as no such key as "cgpa2"

new_item = {"city": "Delhi"}
print(d.update(new_item))
print(d)
```

Loops on Dictionary

We can loop through using key-pair values:

```
d = {
    "name": "Shradha,
    "subjects": ["math", "science", "physics"],
    "cgp a": 9.5
}

for key, value in d.items():
    print(key, value)
```

Sets

What is a Set?

A set is an unordered collection of **unique elements**.
Example:

```
my_set = {1, 2, 2, 2, 3}

print(my_set)           # {1, 2, 3}
print(type(my_set))
print(len(my_set))      # 3


empty_set = set()
```

Set Characteristics

1. Sets can only have **unique** elements.
2. They are **unordered** - no indexing or slicing.
3. They are **mutable** - we can add or remove elements.
4. Set elements must be **immutable** (like strings, numbers, tuples).

Sets are often used when we need uniqueness or mathematical set operations.

Set Methods

Let's have a look at some of the important set methods:

1. `add(val)` - adds an element to set
2. `remove(val)` - removes an element (raises error if not found)
3. `clear()` - removes all elements
4. `pop()` - removes and returns a random element (since sets are unordered)
5. `s1.union(s2)` - returns new union (union is collection of all unique values in both sets)
6. `s1.ntersection(s2)` - returns new union (intersection is collection of all common & unique values in both sets)

```
s = {10, 20, 30}

s.add(40)       # {10, 20, 30, 40}
print(s)

s.remove(10)    # {20, 30, 40}
print(s)

print(s.pop())  # can be any value

s.clear()
print(s)        # set() - empty set

# Union & Intersection
A = {1, 2, 3}
B = {3, 4, 5}

print(A.union(B))         # {1, 2, 3, 4, 5}
print(A.intersection(B))  # {3}
```

> *Keep learning & Keep exploring!*

Python Fundamentals (Part4)

Concepts : Object Oriented Programming in Python

Object Oriented Programming

Let's suppose we need to build a software to store all the information related to students studying in our college.

These students could have a lot of **properties** (like name, parent's name, address, graduation year, cgpa etc.) associated with them.

They could also have a lot of **behaviours** (like fee payment, scholarship calculation, checking minimum attendance criteria etc.) associated with them.

Without existing Python data types, we can choose to store this information in lists or dictionaries but that would not be efficient. As we have a lot of properties, we'd have to create too many lists and it'll be hard to keep track of them. If we use dictionaries, we'd have to re-define the same keys again & again & we won't logically be able to associate behaviours with that data.

So to simplify our work of creating such a software we have **Object Oriented Programming** - which gives us the concept of **classes** & **objects**.

What is a Object Oriented Programming (OOP)?

Object-oriented programming (OOP) in Python is a programming paradigm centered around organizing code into **objects:** bundles of data (attributes) and behaviour (methods). It helps us structure programs in a way that is modular, reusable, and easier to maintain.

Note - It's not compulsory to always use OOP concepts but they definitely help in a lot of cases.

OOP models software as a collection of interacting **objects**, similar to how we think about real-world entities. Each object represents something meaningful in our program - such as a *user*, a *car*, a *bank account*, or an *enemy* in a video game.

In Python, OOP is based around **classes**, which are blueprints for creating objects.

What are Classes & Objects?

Class

A class is a blueprint or template for creating objects. We can think of a class as a recipe: it describes what an object will look like (its attributes) and what it can do (its methods), but it is not the object itself.

```python
class Car:
    brand = "Toyota"
```

Object

An **object** (or **instance**) is a realization of a class, it is the actual thing created based on the class blueprint. Now based on the template we can create as many objects as we want.

```python
car1 = Car()
car2 = Car()

print(car1.brand)      # Toyota
print(car2.brand)      # Toyota
```

We use the "." (Dot Operator) to access properties & methods of objects.

Class vs. Object

Class	Object
Class is a blueprint/ template.	An object is a concrete instance of a class.
Does not exist in memory until instantiated.	Contains actual data & occupies memory.
One class can create any number of objects.	Each object is independent.

Attributes & Methods

Attributes are variables and **Methods** are functions defined inside a class.
(We'll cover both in detail)

Constructor in OOP

In Python, a **constructor** is a special method used to initialize newly created objects. It is not responsible for creating the object, instead the constructor sets up the object with initial values.

We use the `__init__(self, ...)` method to define our constructor. Whenever we create an object of a class, Python automatically calls the `__init__()` method.

```python
class Student:
    def __init__(self):
        print("constructor was called")

stu1 = Student()         # "constructor was called"
```

`self` is a special parameter which refers to the **instance of the class** that is calling the method. We don't need to pass it explicitly.

We can also use constructor to initialize values for objects:

```
class Student:
    def __init__(self, name):
        self.name = name

stu1 = Student("Rahul")
stu2 = Student("Harshita")

print(stu1.name, stu2.name)   # Rahul Harshita
```

Types of Constructors

1. **Default Constructors** - A constructor with **no parameters** except `self`.
2. **Parameterized Constructors** - Takes parameters to initialize values uniquely for each object.

> Note - Python doesn't support constructor overloading directly (like Java/C++) i.e. having multiple constructors in the same class. Whichever is written last is executed.

Attributes in OOP

Attributes are **variables** that belong to a class or an object. They store data/state of the object.

Types of Attributes

1. **Class Attributes**

- Belong to the **class itself**, shared by *all* objects.
- Defined **outside** any method in the class.

```
class Student:
    college = "ABC college"          # class attribute

stu1 = Student ()

print(stu1.college)
print(Student.college) # class attribute can also be accessed with class name
```

2. Instance Attributes

- Belong individually to **each object**.
- Defined inside the `__init__` method using `self` .
- Each object gets its own copy.

```
class Student:
    def _init_(self, name, gpa): # instance attributes
        self.name = name
        self.gpa = gpa

stu1 = Student("Rahul", 8.7)
print(stu1. name, stu1.gpa)
```

Methods in OOP

Methods are **functions defined inside a class**, representing the **behaviour** or **actions** of an object.

Types of Methods

1. Instance Methods

- Take `self` as the first argument.
- Can access both **instance attributes** and **class attributes**.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self. marks = marks

    def display(self):          # Instance method
        print(f"Name: {self.name}, Marks: {self.marks}")
```

2. Class Methods

- Use `@classmethod` decorator.
- Take `cls` (class) as first argument.
- Used to work with **class-level data**.

```
class Student:
    school_name = "ABC School"

    @classmethod
    def change_school(cls, new_name):
        cls.school_name = new_name
```

3. Static Methods

- Use `@staticmethod` decorator.
- Do **not** take `self` or `cls` .
- Behave like normal functions but belong to the class for logical grouping.

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
```

OOP Pillars

Let's understand the 4 Key pillars of OOP - Encapsulation, Abstraction, Inheritance & Polymorphism.

Encapsulation

Encapsulation is the bundling of data (variables) and methods (functions) that operate on that data into a single unit (a class), along with controlling access to that data. This is done to protect the data from accidental or unauthorized modification.

To implement encapsulation, we use access modifiers. Python has 3 access levels:

1. Public members

- Accessible everywhere, written like normal variables.

```
class Student:
    def __init__(self, name):
        self.name = name  # public variable

s = Student("Rahul")
print(s.name)  # Allowed
```

2. Protected members

- Indicated by a **single underscore** `_` (suggest - "Don't access directly unless needed.")
- Still accessible from outside (not truly protected).
- Intended for internal use or inheritance.

```python
class Person:
    def __init__(self):
        self._age = 20  # protected variable

p = Person()
print(p._age)  # Technically allowed, but not recommended
```

3. Private members

- Indicated by a **double underscore** `__`
- Python does *name mangling*: the variable name becomes `_ClassName__variable`.
- Cannot be accessed directly from outside.

```python
class Bank:
    def __init__(self, balance):
        self.__balance = balance # private variable

b = Bank(5000)
print(b.__balance)  # ERROR: attribute not accessible
```

To access:

```python
print(b._Bank__balance) # Allowed (name-mangled form)
```

Python uses naming conventions with access modifiers, not strict enforcement (like Java/C++).

Getters & Setters

When we make a variable private, we use methods to read it (getters) or update it (setters).

```python
class Employee:
    def __init__(self, salary):
        self.__salary = salary       # private

def get_salary(self):                # getter
    return self.__salary

def set_salary(self, new_salary):    # setter
    self.__salary = new_salary

e = Employee(50000)
print(e.get_salary())
e.set_salary(60000)
```

Inheritance

Inheritance is where one class (child) acquires the properties and behaviors (variables + methods) of another class (parent).

The class whose properties are inherited - **Parent** / Base / Superclass

The class that inherits - **Child** / Derived / Subclass

```
class Employee:                # parent class
    start_time = "9AM"
    end_time = "5PM"

class Teacher(Employee):       # child class
    def __init__(self, subject):
        self.subject = subject

t1 = Teacher("Data Science")

print(t1.subject, t1.start_time, t1.end_time)
```

Inheritance enables:

- Code reuse
- Extensibility
- Cleaner, maintainable design
- Polymorphism

Types of Inheritance

1. Single Inheritance

A child inherits from one parent.

```
# Parent → Child

class Parent:
    def display(self):
        print("Parent class")

class Child(Parent):
    pass

c = Child()
c.display() # Output: Parent class
```

2. Multi-level Inheritance

A child inherits from a parent, and another class inherits from the child.

```
# Grandparent(Employee) → Parent(AdminStaff) → Child(Accountant)

class Employee:
    start_time = "9AM"
    end_time = "5PM"

class AdminStaff(Employee):
    def __init__(self, role):
        self.role = role

class Accountant(AdminStaff):
    def __init__(self, salary, role):
        super().__init__(role)
        self.salary = salary

acc1 = Accountant(50_000, "CA")

print(acc1.salary, acc1.role, acc1.start_time, acc1.end_time)
```

3. Multiple Inheritance

A child inherits from more than one parent class.

```
class Teacher:
    def __init__(self, salary):
        self.salary = salary

class Student():
    def __init__(self, gpa):
        self.gpa = gpa

class TA(Teacher, Student):
    def __init__(self, name, salary, gpa):
        super().__init__(salary)            # call parent constructor
        Student.__init__(self, gpa)         # call parent constructor
        self.name = name

ta = TA("Rahul", 50_000, 7.5)

print(ta.name, ta.salary, ta.gpa)
```

super() keyword - Used to call parent class's method from child class.

Abstraction

Abstraction is hiding unnecessary implementation details and showing only the essential features to the user.

Example - In real life when we drive a car & press the breaks, the car stops. But we don't need to know how the hydraulic systems work. To implement same idea in Python, we have abstraction.

We implement abstraction with abstract classes & abstract methods.

Abstract Class

An abstract class in Python is one which:

- Cannot be instantiated
- Can contain normal + abstract methods
- Usually acts as a blueprint for child classes

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass
```

Abstract Method

It is a method declared but not implemented (Children **must** override abstract methods).

```
@abstractmethod
def method_name(self):
```

Example of Abstraction

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound():
        pass

class Lion(Animal):
    def make_sound(self):
        print("Roar!")

class Cow(Animal):
    def make_sound(self):
        print("Moo!")

lion = Lion()
lion.make_sound()

cow = Cow()
cow.make_sound()
```

Polymorphism

Polymorphism is the ability of a single function, operator, or object to behave differently based on the context. (“poly” = many, “morph” = forms)

The idea is that:

- Same method name - works differently for different objects
- Same operator - behaves differently depending on operand types

Let’s look at an example:

```
print(1 + 2)    # adds 2 numbers
print("1" + "2") # concatenates 2 strings
```

Same ‘+’ operator being used for 2 different operations, called **Operator Overloading.**

Let’s look at two popular types of polymorphism:

1. **Function Overriding** (or Method Overriding)

- When a child class provides its own version of a method that already exists in the parent class (Both methods should have same name)
- Type of Runtime Polymorphism (dynamic binding)
- Child method **takes precedence** over parent method.

```
class Animal:
    def sound(self):
        print("Some generic sound")

class Dog(Animal):
    def sound(self):
        print("Bark")

a = Animal()
dog = Dog()

a.sound() # Some generic sound
dog.sound() # Bark
```

2. Duck Typing

- Works on the idea: *"If it looks like a duck and quacks like a duck, it must be a duck."*

```python
class Dog:
    def speak(self):
        print("Bark")

class Cat:
    def speak(self):
        print("Meow")

class Robot:
    def speak(self):
        print("Beep Boop")

def make_it_speak(entity):
    entity.speak()  # doesn't care about type

d = Dog()
c = Cat()
r = Robot()

for e in [d, c, r]:
make_it_speak(e)
```

> *Keep Learning & Keep Exploring!*

Python Fundamentals (Assignment4)

Assignment Problems

> **Concept:** Classes & Objects

Q1. Create a `BankAccount` class with **attributes** account_number, owner_name, and balance.
Add **methods** to deposit, withdraw, and check balance.

> **Concept:** Classes & Objects

Q2. Create a class `Book` with the following attributes:

- title
- author
- list of reviews

And add methods to:

- add a new review
- count reviews
- display all reviews

> **Concept:** Encapsulation

Q3. Create a class `Student` with **private** attributes _name, _roll_no, and _marks. Provide **getter** and **setter** methods with validation (e.g., marks cannot be negative, roll number has to be between 1 & 100 & name cannot be empty).

> **Concept:** Function Overriding

Q4. Create a class `Shape` with a method area().
Create subclasses `Circle`, `Rectangle`, and `Triangle` that **override** the area() method.

> **Concept:** Inheritance

Q5. Create a **base** class `Vehicle` with attributes like brand and model.
Create two **subclasses** `Car` and `Bike` that add extra attributes - seats (in Car) & engine_cc (in Bike).

> **Concept:** Abstraction

Q6. Create an **abstract** class `Employee` with an **abstract method** calculate_salary().
Create subclasses `Intern`, `FullTimeEmployee`, and `ContractEmployee` that implement the method differently.

> **Concept:** Constructor Overloading (with Default Parameters)

Q7. Create a class `Person` that allows the constructor to work with:

- name only
- name + age
- name + age + address

As direct constructor overloading (multiple constructors) are not allowed but we have to use **default parameters** to simulate constructor overloading.

> **Concept:** Instance & Class Attributes

Q8. Create a class `Player` with:

- a class variable player_count
- instance variables name and level

Track how many players were created.

> **Concept:** Multiple Inheritance

Q9. Create the following classes: `Herbivore`, `Carnivore`, `Omnivore` with some attributes & methods. Then create a class `Bear` that inherits from all the above classes to showcase how multiple inheritance works.

> **Concept:** OOP

Q10. Mini Project – OOP Chat System

Let's create a Chat System using OOPs concepts. We have to create classes:

- `User`
- `Message`
- `ChatRoom`

And we have to implement functions:

- sending messages
- viewing chat history
- user joining and leaving the chatroom

Chat system - code

```
# ------------------------------
# Message Class
# ------------------------------
class Message:
    message_counter = 1   # simple counter

    def __init__(self, sender, content):
        self.sender = sender
        self.content = content
        self.id = Message.message_counter
        Message.message_counter += 1

    def __str__(self):
        return f"({self.id}) {self.sender.username}: {self.content}"


# ------------------------------
# User Class
# ------------------------------
class User:
    def __init__(self, username):
        self.username = username
        self.chatroom = None

    def join_chatroom(self, chatroom):
        if self.chatroom:
            print(f"{self.username} is already in a chatroom.")
        else:
            chatroom.add_user(self)
            self.chatroom = chatroom
            print(f"{self.username} joined {chatroom.name}")

    def leave_chatroom(self):
        if not self.chatroom:
```

```
            print(f"{self.username} is not in any chatroom.")
        else:
            self.chatroom.remove_user(self)
            print(f"{self.username} left {self.chatroom.name}")
            self.chatroom = None

    def send_message(self, content):
        if not self.chatroom:
            print(f"{self.username} cannot send a message (not in a chatroo
m).")
        else:
            self.chatroom.broadcast(self, content)


# ------------------------------
# ChatRoom Class
# ------------------------------
class ChatRoom:
    def __init__(self, name):
        self.name = name
        self.users = []
        self.messages = []

    def add_user(self, user):
        self.users.append(user)

    def remove_user(self, user):
        self.users.remove(user)

    def broadcast(self, sender, content):
        message = Message(sender, content)
        self.messages.append(message)
        print(message)   # Show message to all users

    def show_chat_history(self):
        print(f"\nChat History of {self.name}:")
        for msg in self.messages:
            print(msg)
```

```
        print()


# ---------------------------------------
# Example Usage
# ---------------------------------------
if __name__ == "__main__":
    room = ChatRoom("Python Lounge")

    u1 = User("Alice")
    u2 = User("Bob")
    u3 = User("Charlie")

    u1.join_chatroom(room)
    u2.join_chatroom(room)

    u1.send_message("Hello everyone!")
    u2.send_message("Hi Alice!")

    u3.join_chatroom(room)
    u3.send_message("Hey guys, what's up?")

    room.show_chat_history()

    u1.leave_chatroom()
    u2.leave_chatroom()
    u3.leave_chatroom()
```

Python Fundamentals (Part5)

Concepts : File I/O, Exception Handling, List Comprehensions, JSON module

File I/O

File I/O (Input/Output) means **reading data from files** and **writing data to files** using Python.
Python provides built-in functions to work with files.

Opening a File

We use the `open()` function:

```
file = open("filename", "mode")
```

Example

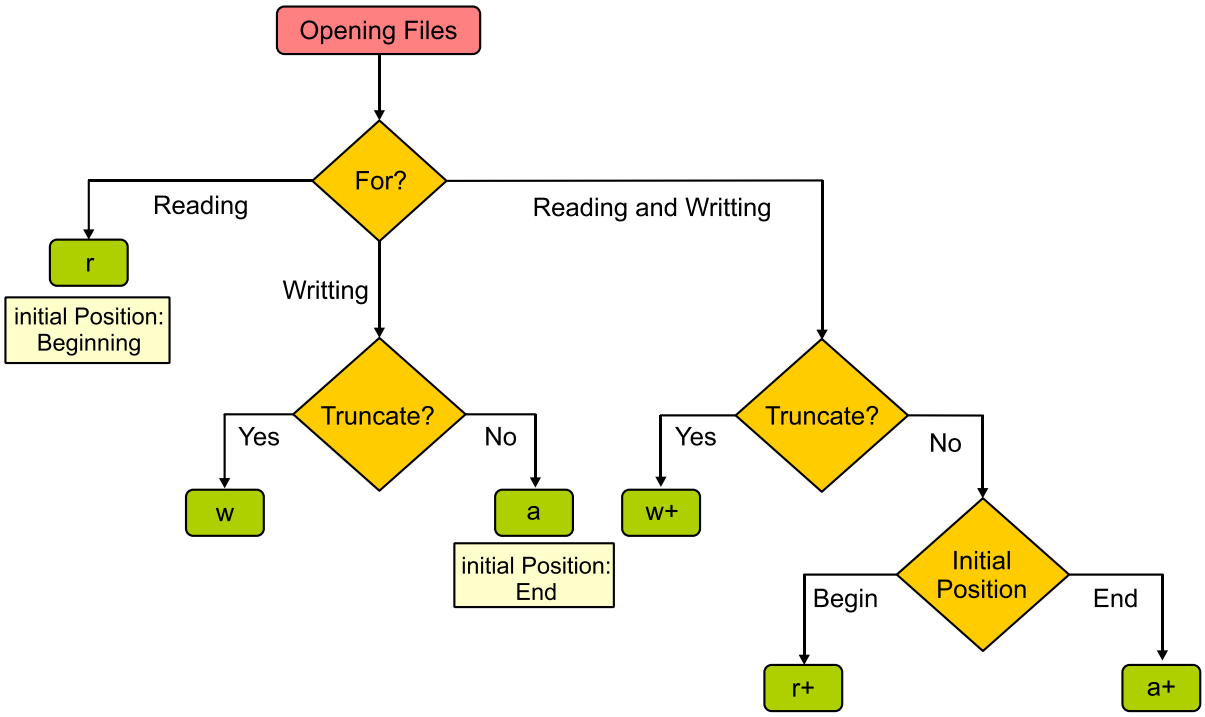
```
f = open("data.txt", "r")
```

Common Modes

Mode	Meaning	Description
`r`	Read	Opens file for reading (default). Error if file doesn't exist.
`w`	Write	Creates new file or overwrites existing file.
`a`	Append	Adds content at the end of the file.
`r+`	Read + Write	File must exist.
`w+`	Write + Read	Overwrites existing file.
`a+`	Append + Read	Reads + adds to end of file.
`b`	Binary Mode	For images, videos, etc. (use with other modes like `rb`).

What to use when?

Follow this flowchart:



Reading from a File

We have multiple functions to read content from a file.

1. `read()`

   Reads entire file as a single string.

```
f = open("data.txt", "r")
content = f.read()
print(content)
f.close()
```

2. `readline()`

   Reads one line at a time.

```
f = open("data.txt", "r")
line1 = f.readline()
line2 = f.readline()
f. close()
```

3. `readlines()`

Reads all lines into a list.

```
f = open("data.txt", "r")
lines = f.readlines()
```

Writing to a File

1. `write()`

Writes a string to a file.

```
f = open("data.txt", "w")
f.write("Hello students!")
f.close()
```

2. `writelines()`

Writes multiple lines at once.

```
f = open("data.txt", "a")
f.writelines(["Line 1\n", "Line 2\n"])
f.close()
```

Always remember to close a file at the end to free system resources.

Recommended: Usage `with open()`

We use a context manager that automatically closes the file.

```
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

Deleting a File

To delete a file we use the `remove` function.

```
import os

os.remove("data.txt")
```

Exception Handling

An **exception** is an error that occurs while a program is running. If not handled, the program crashes. Exception Handling allows you to manage errors gracefully so your program continues to run.

An exception occurs when Python encounters something it cannot handle during execution.

Examples:

- Dividing by zero - `ZeroDivisionError`
- Using an undefined variable - `NameError`
- Opening a missing file - `FileNotFoundError`
- Wrong data type - `TypeError`

Basic Syntax

```
try:
    # Code that may cause an error
except:
    # Code that runs if error occurs
```

Example:

```
try:
    x = 10 / 0
except:
    print("Error occurred!")
```

We can also throw specific exceptions:

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("You can't divide by zero!")
```

The `else` Block

`else` executes only if no exception happens.

```
try:
    x = int(input("Enter a number: "))
except ValueError:
    print("Invalid input!")
else:
    print("You entered:", x)
```

The `finally` Block

`finally` always executes, whether an exception occurs or not. It is used for cleanup tasks (closing files, releasing resources).

```
try:
    f = open("data.txt")
    print(f.read())
except FileNotFoundError:
    print ("File not found!")
finally:
    print("Execution completed.")
```

List Comprehensions

List Comprehension is a **short and elegant way** to create lists in Python.
It replaces long `for` loops with **one-line expressions**.

Basic Syntax

```
[expression for item in iterable]
```

Example: Create a list of numbers 1 to 5

```
nums = [x for x in range(1, 6)]
print(nums)     # [1, 2, 3, 4, 5]
```

List Comprehension with Condition

```
[expression for item in iterable if condition]
```

Example: Even numbers from 1 to 10

```
evens = [x for x in range(1, 11) if x % 2 == 0]
```

List Comprehension with if-else

```
[expression_if_true if condition else expression_if_false for item in iterable]
```

Example: Label numbers as “Even” or “Odd”

```
labels = ["Even" if x % 2 == 0 else "Odd" for x in range(1, 6)]
```

Working with JSON Module

JSON (**J**ava**S**cript **O**bject **N**otation) is a lightweight data format used to exchange data between programs, APIs, websites, etc. JSON format is very similar to Python dictionaries.

Python’s `json` module allows you to read, write, encode, and decode JSON.

Importing the Module

```
import json
```

Converting Python to JSON

We use `dumps()` to convert Python objects into JSON string.

```
data = {
    "name": "John",
    "age": 25,
    "marks": [85, 90, 92]
}

json_string = json.dumps(data)
print(json_string)
```

Converting JSON to Python

We use `loads()` to convert JSON string to Python dictionary.

```
json_data = '{"name": "John", "age": 25}'
python_obj = json.loads(json_data)
print(python_obj["name"])
```

Reading JSON from a File (`json.load`)

```
import json

with open("data.json", "r") as f:
    data = json.load(f)

print(data)
```

Writing JSON to a File (`json.dump`)

```
import json

data = {"name": "Aisha", "city": "Delhi"}

with open("data.json", "w") as f:
    json.dump(data, f, indent=4)
```

`indent=4` makes the JSON looks neat and readable.

> *Keep Learning & Keep Exploring!*

NumPy

What is NumPy?

NumPy (short form of **Numerical Python**) is an open-source Python library used for numerical computing. It provides tools for working efficiently with arrays, matrices, and mathematical functions.

NumPy introduces a new data structure called the **ndarray** (n-dimensional array), which is much faster and more memory-efficient than Python's built-in lists for numerical operations.

Important Features

- **Multidimensional Arrays:**
  Efficient storage and manipulation of large datasets (e.g., 1D, 2D, 3D arrays).
- **Mathematical Functions:**
  A wide range of mathematical operations — linear algebra, Fourier transforms, random number generation, etc.
- **Vectorization:**
  Operations on entire arrays without explicit loops, making code cleaner and faster.
- **Interoperability:**
  Works well with other libraries like Pandas, Matplotlib, TensorFlow, PyTorch etc.

Usage

```
import numpy as np

# Create & Output an array
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

NumPy Arrays vs Python Lists

1. NumPy arrays are faster than Python lists (implemented in C, not pure Python).
2. They store elements in **contiguous memory** (better cache performance).
3. They are also homogenous i.e. all elements have same type.
4. They use **vectorized operations** (no slow Python loops).

Performance Comparison

```
import numpy as np
import time

size = 10_000_000 # large data set of 10 million numbers

# Python Lists
python_list = list(range(size))
start = time.time()

list_squared = [x**2 for x in python_list] # square of all nums

end = time.time()
print("Python list time:", end - start, "seconds")

# NumPy Arrays
np_array = np.array(python_list)
start = time.time()

array_squared = np_array ** 2 # vectorized operation
end = time.time()

print("NumPy array time:", end - start, "seconds")
```

Memory Usage Comparison

```
# Memory Usage comparison
import sys

print("Python list size:", sys.getsizeof(python_list) * len(python_list))
print("NumPy array size:", np_array.nbytes)
```

Creating NumPy Arrays

There are multiple ways of creating NumPy arrays, most common of which are:

1. From Python Lists

```
# Creating NumPy Arrays - from lists
arr = np.array([1, 2, 3, 4])
print(arr, type(arr))

arr2 = np.array([1, 2, 3, 4, "prime", 3.14])
print(arr2, type(arr2))

# 2D Arrays - Matrix
arr3 = np.array([[1, 2, 3], [4, 5, 6]])
print(arr3, arr3.shape)
```

Note - All elements in `arr2` in above code will have same type (homogenous) unlike lists.

2. Using built-in Functions

```
# Creating NumPy Arrays - from scratch
arr1 = np.zeros((3, 4)) # 3x4 array of 0s
print(arr1, arr1.shape)

arr2 = np.ones((3, 3)) # 3x3 array of 1s
print(arr2, arr2.shape)

arr3 = np.full((2, 3), 5) # 2x3 array of 5s
print(arr3, arr3.shape)

arr4 = np.eye(3) # Identity matrix of 3x3
print(arr4, arr4.shape)

arr5 = np.arange(1, 20, 2) # Elements in range(1, 20)
print(arr5, arr5.shape)

arr6 = np.linspace(0, 10, 5) # Evenly spaced array
print(arr6, arr6.shape)
```

NumPy Array Properties

Array properties helps you understand and manipulate data in arrays efficiently.

```
# Useful Attributes
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])

print(arr.shape) # Dimensions - (4 x 3)
print(arr.size)  # Total elements - (12)
print(arr.ndim)  # Number of dimensions - 2
print(arr.dtype) # Data type object - int64
print(arr.itemsize) # Size of each element in bytes - 8 for int64
```

We can also explicitly change the `dtype` for our arrays.

```
# Specify dtype at creation
str_arr = np.array([1, 2, 3], dtype="U")
print(str_arr, str_arr.dtype)

float_arr = np.array([1, 2, 3], dtype="float64")
print(str_arr, float_arr.dtype)

# Creating new array with a specific type from existing array
int_arr = float_arr.astype(np.int64)
print(int_arr, int_arr.dtype)
```

Operations on Arrays

There are a lot of useful operations that we can perform on our arrays.

1. Reshaping

```
arr = np.array([1, 2, 3, 4, 5, 6])
print(arr.shape)

reshaped = arr.reshape((2, 3)) # converts (1x6) => (2x3)
print(reshaped, reshaped.shape)

flattened = reshaped.flatten() # converts 2D => 1D
print(flattened, flattened.shape)
```

2. Indexing

```
# Indexing for 1D array
arr = np.array([1, 2, 3, 4, 5])
print(arr[0])

# Indexing for 2D array
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]) # 2D array
print(arr[0][1]) # 2
print(arr[1][2]) # 6
```

Apart from simple indexing, we can also use Fancy & Boolean indexing.

Fancy indexing means accessing array elements using integer arrays/lists of indices rather than plain slices (`:`).

```
# Fancy Indexing
arr = np.array([1, 2, 3, 4, 5])
idx = [0, 1, 4]
print(arr[idx])        # print nums at given indices
```

Boolean masking(or indexing) means using a Boolean array (`True`/`False`) to select elements from another array.

```
# Boolean Indexing
print(arr[arr > 2])          # print nums greater than 2
print(arr[arr % 2 == 0])     # print even num
```

3. Slicing

```
# Slicing 1D array
arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[2:6]) # [3, 4, 5, 6]
print(arr[:6])  # [1, 2, 3, 4, 5, 6]
print(arr[3:])  # [4, 5, 6, 7]
print(arr[::2]) # [1, 3, 5, 7]
```

Copy v/s View

Slicing a list returns a copy but slicing a NumPy array returns a view - for efficiency. View is like a shallow copy that shares the same data as the original array, so no duplication happens here.

- **Views** are **fast** and **memory-efficient** (no data duplication).
- **Copies** are **safe** but **slower** and use **more memory**.

```
# Sliced List is a COPY
py_list = [1, 2, 3, 4, 5]
copy_list = py_list[1:4] # [2, 3, 4]
copy_list[1] = 333

print(copy_list)
print(py_list) # [1, 2, 3, 4, 5] - remains same

# Sliced Array is a VIEW
np_arr = np.array([1, 2, 3, 4, 5])
view_arr = np_arr[1:4] # [2, 3, 4]
view_arr[1] = 333

print(view_arr)
print(np_arr) # [1, 2, 333, 4, 5] - changes

# Creating a COPY for Array
copy_arr = np_arr[1:4].copy() # [2, 3, 4]
copy_arr[2] = 444
print(copy_arr)
print(np_arr) # [1, 2, 3, 4, 5] - remains same
```

Multi-dimensional Arrays

Multi-dimensional arrays in NumPy are the foundation of most scientific and machine-learning work.
A NumPy array can have **any number of dimensions** (1D, 2D, 3D & so on). Each dimension is called an **axis**.

- 1D array has 1 axis (axis0).
- 2D array has 2 axes (axis0 = rows, axis1 = columns)
- 3D array has 3 axes (axis0 = depth/layer, axis1 = rows in each layer, axis2 = columns in each layer)

```
# 1D array
arr1D = np.array([1, 2, 3])
print(arr1D.ndim) # 1

# 2D array (matrix)
arr2D = np.array([[1, 2, 3],
                  [4, 5, 6]])
print(arr2D.ndim) # 2

# 3D array (tensor)
arr3D = np.array([[[1, 2, 3],
                   [4, 5, 6]],
                  [[7, 8, 9],
                   [10, 11, 12]]])
print(arr3D.ndim) # 3
```

Usage of multi-dimensional arrays in Practice

We'll look into a practical example of dealing with real-world images data. If we have a ML/DL model working with images, then we can feed this data as 2D or 3D arrays.

1. **Grayscale Image** as 2D array
   A grayscale image has **height × width** pixels. Each pixel has a single intensity value (0–255 for 8-bit images).
2. **Color Images** as 3D array
   A color image(RGB) has **height × width × channels**. The 3 channels are → RGB (Red, Green, Blue) & each channel is a 2D array of pixel intensities (0-255).

We'll cover practical implementation in later chapters.

Operations along Axes

We can also perform certain **operations along a specific axis** in an array.

```
arr2D = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print(np.sum(arr2D)) # sum of entire array - 45

sum_of_columns = np.sum(arr2D, axis = 0)
print(sum_of_columns) # [12 15 18]

sum_of_rows = np.sum(arr2D, axis = 1)
print(sum_of_rows) # [6 15 24]

# Slicing
print(arr2D[0:3, 1:3]) # slice rows(0, 1, 2) x cols(1, 2)
# [[2 3]
# [5 6]
# [8 9]]
```

Operations on 3D arrays

Let's look at how do we work with **3D** arrays:

```
arr3D = np.array([[[1, 2],[3, 4],[5, 6]], [[7, 8],[9, 10],[11, 12]]])

print(arr3D, arr3D.shape)

# Indexing
print(arr3D[0][1][1]) # 4
print(arr3D[1][2][1]) # 12

print(arr3D[:, :, 0])  # first col from both layers
print(arr3D[:, 0, :])  # first row from both layers

# Manipulating data
arr3D[:, 0, :] = 99    # change first row to store 99
print(arr3D)
```

Data Types in NumPy

We have already discussed how every NumPy array has a **single data type** (homogeneous arrays) & it is stored in the `.dtype` attribute.

Now, let's look into some of the most **common data types** in NumPy:

- Integer literals : `int32` , `int64`
- Floating literals : `float32` , `float64`
- Boolean : `bool`

- Complex numbers : `complex64` , `complex128`
- String : `S` (byte-str) & `U` (unicode-str)
- Object : generic python objects – `object`

```
# Common Data Types
arr = np.array([1, 2, 3, 4, 5])
arr2 = np.array([1.0, 2.0, 3.0])
arr3 = np.array(["hello", "world", "prime", "ai/ml"])

print(arr.dtype) # int64
print(arr2.dtype) # float64
print(arr3.dtype) # U

# Complex Numbers
arr1 = np.array([2 + 3j])
arr2 = np.array([5 + 8j])

print(arr1, arr1.dtype)
print(arr1 + arr2)
print(arr2 - arr1)

# Objects
arr = np.array (["hello", {1, 2, 3}, 3.14])
print(arr, arr.dtype)
```

We can also change the data type, either by explicitly typecasting at array creation using `dtype` attribute or by using the `astype` method while creating a new array from existing one.

```
# Changing the data type
new_arr = arr.astype("float64")
print(new_arr, new_arr.dtype)

new_arr = np.array([1, 2, 3, 4, 5], dtype="float64")
print(new_arr, new_arr.dtype)
```

Why does dtype matter?

- **Memory efficiency**
  - `np.int8` uses 1 byte per element, `np.int64` uses 8 bytes.
- **Performance**
  - Smaller types = faster computations.
- **Compatibility**
  - Images often use `np.uint8`
  - ML libraries expect `float32`

Note - In some case it is useful to do **Downcasting** i.e. converting type to a smaller data type. This is done to reduce memory usage & improve performance.

Example - Suppose you have a dataset of 1 million people's ages. Storing them as `int64` wastes memory because ages are small numbers (0–120). So we can downcast these values to `Int8`.

Vectorization & Broadcasting in NumPy

Vectorization and **Broadcasting** in NumPy are two of the most powerful features for fast numerical computations.

Vectorization

Vectorization means performing **operations on entire arrays at once** without explicit Python loops.

- NumPy uses **C-level implementations** internally → much faster than Python loops.
- Makes code **shorter, cleaner, and faster**.

```
arr = np.array([1, 2, 3, 4, 5])

sq_arr = arr ** 2        # Square of all nums
print(sq_arr)

arr2 = np.array([6, 7, 8, 9, 10])
print(arr + arr2)        # Sum of 2 arrays
```

Broadcasting

Broadcasting allows NumPy to **automatically expand arrays of different shapes** so that arithmetic operations can be performed. It's basically scaling arrays without using extra memory.

- No need to manually reshape arrays.
- Useful for **combining arrays of different dimensions**.

Broadcasting Rules

Broadcasting can only take place when the arrays are of compatible shape. So NumPy compares shapes of arrays **from right to left**. For the array to be compatible, all dimensions must either be:

- **Equal**, or
- **1**, or
- **Missing** (smaller array can be “stretched”).

```
# Broadcasting with a Scalar
arr_mul10 = arr * 10 # Multiply by 10 to all nums
print(arr_mul10)

# Broadcasting with a Vector
arr1D = np.array([1, 2, 3])
arr2D = np.array([[1, 2, 3], [4, 5, 6]])
print(arr1D + arr2D)
```

A quite common example of broadcasting in **Vector Normalization**. This is very common in machine learning and data preprocessing.

Let’s take an example of **Standard Vector Normalization** i.e. transforming an array such that it has:

- mean = 0
- standard deviation = 1

For each element x_i in vector, $x_i^{\text{normalized}} = (x_i - \mu)/\sigma$

Where:

- μ = mean of the vector (or column)
- σ = standard deviation

```
# Standard Vector Normalization
arr = np.array([[1, 2], [3, 4]])
mean = np.mean(arr)
std_dev = np.std(arr)
normalized_arr = (arr - mean) / std_dev

print(normalized_arr)

# Column wise Normalization
arr = np.array([[1, 2], [3, 4], [5, 6]])
mean = np.mean(arr, axis = 0)
std_dev = np.std(arr, axis = 0)
print((arr - mean) / std_dev)
```

Extra references: understand Standard Deviation and Variance.

Useful Mathematical Functions

NumPy is massive & provides a **wide range of built-in mathematical functions** that are highly optimized and can operate **element-wise on arrays**. Let's have a look at some of them:

Aggregation Functions

These are functions that take an array and reduce it to a single value (or smaller array) by combining elements.

1. `sum()` - returns sum of all elements
2. `prod()` - returns product of all elements
3. `min()` - returns minimum value
4. `max()` - returns maximum value
5. `argmin()` - returns index of min value
6. `argmax()` - returns index of max value
7. `mean()` - returns mean (average)
8. `median()` - returns median
9. `std()` - returns standard deviation
10. `var()` - returns variance

```
arr = np.array([1, 2, 3, 4, 5])

print(np.sum(arr))      # 15
print(np.prod(arr))     # 120
print(np.min(arr))      # 1
print(np.argmin(arr))   # 0
print(np.max(arr))      # 5
print(np.argmax(arr))   # 4
print(np.mean(arr))     # 3.0
print(np.median(arr))   # 3.0
print(np.std(arr))      # 1.41
print(np.var(arr))      # 2.0
```

Power Functions

1. `square()` - returns square of all elements
2. `sqrt()` - returns square root of all elements
3. `pow(a, b)` - returns a^b

```python
arr = np.array([1, 2, 3, 4, 5])

print(np.square(arr))  # [1, 4, 9, 16, 25]
print(np.sqrt(arr))    # [1, 1.41, 1.73, 2, 2.23]
print(np.pow(arr, 3))  # [1, 8, 27, 64, 125]
```

Log & Exponential Functions

1. `log()` - returns natural log
2. `log10()` - returns log base 10
3. `log2()` - returns log base 2
4. `exp(x)` - returns e^x

```python
print(np.log(arr))
print(np.log10(arr))
print(np.log2(arr))
print(np.exp(arr))
```

Rounding Functions

1. `round()` - rounds off to nearest value
2. `ceil()` - rounds up
3. `floor()` - rounds down
4. `trunc(x)` - truncates(removes) the fractional part

```python
print(np.round(2.678)) # 3.0
print(np.floor(2.678)) # 2.0
print(np.ceil(2.678))  # 3.0
print(np.trunc(2.678)) # 2.0
```

Miscellaneous Functions

1. `abs()` - returns absolute value
2. `sort()` - returns sorted(arranges in increasing order) values
3. `unique()` - returns unique values

```python
arr = np.array([1, 2, -5, 3, 8, -4, 2, 5])

print(np.abs(arr))     # [1 2 5 3 8 4 2 5]
print(np.sort(arr))    # [-5 -4 1 2 2 3 5 8]
print(np.unique(arr))  # [-5 -4 1 2 3 5 8]
```

There are many other built-in functions that we will use and learn about in later chapters.

> *Keep Learning & Keep Exploring!*

Pandas

What is Pandas?

Pandas is one of the most widely used Python libraries for **data analysis and manipulation**. It is open-source and built on the top of **NumPy**. It adds high-level data structures and tools that make it easier to work with **tabular**, **labeled**, or **heterogeneous** datasets.

Data Structures

Pandas introduces two important data structure: Series & DataFrame.

Usage

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Alice", "Bob"],
    "Cgpa": [9.5, 8.7]
})
```

Core Data Structures in Pandas

Series

A Series is a **one-dimensional labeled array** (like a column in a spreadsheet). It can hold data of any type: integers, floats, strings, Python objects.

It has two main components:

1. **Values** - the actual data
2. **Index** - labels for each value

Usage

```
s = pd.Series([23, 24, 25, 26])
print(s)
print(type(s))

# Indexing
print(s[0])      # 23
print(s[2])      # 25

print(s.index)       # all labels
```

Characteristics of a Series

1. They are Homogeneous - store one type of data.
2. They support Vectorized operations.
3. They can handle missing values with NaN.
4. They have mutable values but immutable size.

```
# Custom Indexing
s2 = pd.Series([23, 24, 25, 26], index = ["Adam", "Eve", "Charlie", "Bob"])
print(s2["Eve"])    # 24
print(s2["Bob"])    # 26

# Vectorized Operations
s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])

print(s1 + s2)

# Mutable Values but immutable size
s = pd.Series([1, 2, 3, 4, 5])
s[0] = 100

print(s)

changed_s = s.drop(1)
print(changed_s)
print(s)
```

DataFrame

A DataFrame is a **two-dimensional, tabular data structure** (like a spreadsheet or SQL table).

It consists of:

- Rows
- Columns
- Index (row labels)
- Column labels

Each column in a DataFrame is a Series.

Usage

```
# Creating DataFrame in pandas - using dictionary
info = {
    "Name" : ["Adam", "Eve", "Bob"],
    "Marks" : [78, 99, 85],
    "Grade" : ['B', 'O', 'A']
}

df = pd.DataFrame(info)

print(df)
print(type(df))

print(df.index)         # row labels
print(df.columns)       # column labels

# Creating DataFrom using Numpy array
np_arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
df = pd.DataFrame(np_arr, columns=["Col1", "Col2", "Col3"])
print(df)

# Creating DataFrom using Lists
l = [["Adam", 96], ["Eve", 75], ["Bob", 82], ["Charlie", 92]]
df = pd.DataFrame(l, columns=["Name", "Marks"])
print(df)
```

Working with Data Files

Most of the time when we deal with data we'll get it in a format like **CSV** (Comma-Separated Values) or **JSON** (JavaScript Object Notation) and Pandas makes it extremely easy to read data from various file formats.

Importing Data

```
df_csv = pd.read_csv("data.csv")     # importing data from csv file

df_json = pd.read_json("data.json")  # importing data from json file
```

Exporting Data

```
df.to_csv("output.csv")   # exporting to csv

df.to_csv("output.csv", index=False)  # exporting without index

df.to_json("output.json")   # exporting to json
```

DataFrame Methods

We have a lot of useful methods with DataFrame.

Data Viewing & Inspection

These help us take a quick look at our dataset.

1. `df.head(n)` - Shows the first n rows (default = 5)
2. `df.tail(n)` - Shows the last n rows (default = 5)
3. `df.sample(n)` - Shows random n rows (default = 1)
4. `df.info()` - Displays column names, data types, memory usage
5. `df.describe(n)` - Shows descriptive statistics for numeric columns.
6. `df.nunique(n)` - Shows count of distinct values exist in each column.

We also have attributes like:

1. `df.shape` - Returns (rows, columns).
2. `df.columns` - List of column names.
3. `df.dtypes` - Data types of each column.

```
data = {
    'Name': ['Aarav', 'Isha', 'Rohan', 'Sneha', 'Vikram'],
    'Age': [25, 30, 35, 40, 45],
    'City': ['Delhi', 'Mumbai', 'Bangalore', 'Kolkata', 'Chennai']
}

df = pd.DataFrame(data)

print(df.head())
print(df.tail())
print(df.sample())
print(df.info())
print(df.describe())
print(df.nunique())

print(df.dtypes)
print(df.shape)
print(df.columns)
```

Indexing & Data Selection

- **Selecting Columns**

  We can select by column name (as a Series) or multiple columns at once.

```
df["country"]        # returns a Series
df[["city", "aqi"]]  # returns a DataFrame
```

- **Selecting Rows**

  By index label (`loc`) - Use row labels (if you have custom index) or column names.

```
df.loc[0]              # 1st row (by label)
df.loc[0:3]            # row0 to row3 - both inclusive in loc

df.loc[0, "country"]   # 0th row & specific column
```

  Note : In `loc`, when we slice, the ending index is inclusive.

By index position (`iloc`) - Use integer positions.

```
df.iloc[0]              # 1st row
df.iloc[4:7]            # 1st row

df.iloc[0, 2]           # 1st row & 3rd column (by position)
```

Note : In `iloc`, when we slice, the ending index is NOT inclusive.

- **Selecting single Cell**

  We have `at` & `iat` for fast access to single cells.

```
# Select single scalar value
df.at[0, "city"]
df.iat[0, 2]
```

Filtering & Querying Data

- **Boolean Filtering**

  We can filter a DataFrame using a condition that returns a boolean Series.

```
df[df['Age'] > 30]
df[(df['Age'] > 30) & (df['Salary'] > 50000)]
```

  We can use `&` for AND, `|` for OR & wrap each condition in parentheses `()`.

- `df.query()` **Method**

  Query method provides us with SQL-like filtering. We can pass our query in a string.

```
# Example 1
df.query("Age > 30 and Salary < 70000")

# Example 2
my_country = "IN"
df.query("country == @my_country")
```

  Query returns a COPY, not a VIEW.

Query String Rules:

1. We write the condition in a string.
2. We can use operators like `and`, `or`, `not`, `==`, `!=`, `>`, `<`, `>=`, `<=` etc.
3. Use backticks for column names with spaces or symbols.
4. Use @ for Python variables.

Important - Rule of thumb:

- Direct filtering using df[...] - prefer using `&` / `|` for AND & OR.
- Inside `query()` strings - prefer using `and` / `or` for AND & OR.

Data Cleaning

- **Handle Missing Values (`NaN`)**

1. `df.isnull()` - Shows boolean DataFrame, True where NaN
2. `df.isnull().sum()` - Count of NaNs per column
3. `df.dropna()` - Drop rows with any NaN
4. `df.fillna(val)` - Fills NaN with a value
5. `df.ffill()` - Forward Fill (carry previous value)
6. `df.bfill()` - Backward Fill (carry next value)

```
# Handle Missing values
df.isnull()             # True for null values (NaN)
df.isnull().sum()       # counts missing vals per column

df.dropna()             # drops rows with missing values
df.dropna(axis = 1)     # drops cols with missing values

df.fillna(0)            # fills NaN with 0
df["age"] = df["age"].fillna(df["age"].mean())    # fills column with mean

df.ffill()              # forward fill
df.bfill()              # backward fill
```

- **Handle Duplicate Values (`NaN`)**
  1. `df.duplicated()` - Find duplicates
  2. `df.drop_duplicates()` - Remove duplicate rows

```
df.duplicated()               # True for duplicate rows
df.duplicated("country")      # True for duplicate rows in particular col
df.duplicated(["country", "gender"])

df.drop_duplicates()                # drops duplicate rows
```

- **Changing Data Types**
  1. `df.astype(new_type)` - Changes dtype.
  2. `pd.to_datetime()` - To convert a string, number, or array-like object to a datetime object.

```
df2["income"] = df2["income"].astype(int)         # change dtype

date_str = "2025-12-01"
date = pd.to_datetime(date_str)
print(date)
print(type(date))
```

- **String Cleaning**
  1. `.str.lower()` - Convert to lowercase
  2. `.str.upper()` - Convert to uppercase
  3. `.str.capitalize()` - Capitalize strings
  4. `.str.strip()` - Remove leading/trailing spaces
  5. `.str.split(" ")` - Split into parts based on a separator
  6. `.str.contains()` - Check if a value exists in string or not

```
df2["gender"].str.lower()                    # lower case
df2["gender"].str.upper()                    # upper case
df2["gender"].str.capitalize()               # capitalize

df2["gender"].str. capitalize()              # capitalize
df2["name"].str.split(" ")                   # splits into lists
type(df2["name"].str.split(" ")[0])

df2["country"].str.contains("US")
df2["country"].str.contains("india", case=False)
```

Transforming Data

- **Applying Functions**

1. `apply()` - Apply a function to a Series or DataFrame.

```
df['Age_plus_10'] = df['Age'].apply(lambda x: x + 10)
```

2. `map()` - Map a function or dictionary to a Series.

```
df['City'] = df['City'].map({'Delhi':'DL', 'Mumbai':'MB'})
```

3. `assign()` - Create new columns or modify existing columns.

```
df.assign(new_income = df["income"] * 1.1)
```

4. `replace(old, new)` - Replace specific values in a Series or DataFrame.

```
df['City'] = df['City'].replace({'Delhi':'DL', 'Mumbai':'MB'})
```

- **Renaming / Reordering**

1. `rename()`

```
df = pd.DataFrame({
    "A": [1, 2, 3],
    "B": [4, 5, 6],
    "C": [7, 8, 9]
})

# Rename columns A -> X, B -> Y
df_renamed = df.rename(columns={"A": "X", "B": "Y"})
print(df_renamed)
```

2. Reorder columns

```
df.columns = ["X", "Y", "Z"]  # reorders all at once

# Reorder columns to C, A, B
df_reordered = df[["C", "A", "B"]]
```

The list must include all columns we want to keep & columns not listed will be dropped.

3. Reset Index

```
sorted_df2.reset_index()
sorted_df2.reset_index(drop=True)    # to drop original index vals
```

- **Sorting / Ranking**

1. `sort_values()` - Sort values in ascending or descending order.
2. `sort_index()` - Sort indexes in ascending or descending order.
3. `rank()` - Sort values in ascending or descending order, `method` resolves the ties.

```
# Sorting - values & index
df2.sort_values("Income")                  # sort values in ascending
df2.sort_values("Income", ascending=False) # sort values in descending
df2.sort_values(["Age", "Income"]) # sorts age,if age same then sorts income

sorted_df2 = df2.sort_values(["Age", "Income"])
sorted_df2.sort_index()

# Ranking
df2["Ranking"] = df2["Income"].rank(ascending=False, method="dense")
df2["Ranking"] = df2["Income"].rank(ascending=False, method="min")
df2["Ranking"] = df2["Income"].rank(ascending=False, method="max")
```

Grouping & Aggregation Methods

- `df.groupby()`

  We use the `groupby` method to split data into multiple groups so that we can apply some aggregation functions on it.

  ```
  df2.groupby("country")["income"].mean()    # mean income for each country
  ```

- Aggregations
  - `df.sum()`
  - `df.mean()`
  - `df.count()`
  - `df.max()`
  - `df.min()`
  - `df.std()`

  ```
  df2.groupby("gender")["income"].mean()  # mean income for each gender
  df2.groupby("country")["gender"].count() # count of gender for each country
  df2.groupby("gender")["income"].max()   # max income for each gender
  ```

- `df.aggr()`

  We use the `aggr` or `aggregation` function for multiple aggregations i.e. on multiple columns.

```python
# applies multiple aggregate functions
df.groupby("country")["income"].agg(["mean", "min", "max"])
df.groupby("country")["income"].aggregate (["mean", "min", "max"]) # alias

# rename aggregate
df.groupby("country")["income"].agg(avg_salary="mean", max_salary="max")

df.groupby("country").agg({
    "age": "mean",
    "income": "mean"
}) # aggregate on multiple cols

df.groupby("country").agg(
    avg_age=("age", "mean"),
    avg_salary= ("income", "mean")
) # rename aggregates on multiple cols
```

Reshaping Methods

We have two important methods to reshape our data - Melt & Pivot.

- **Melt (wide → long)**
  - Convert a wide format DataFrame into a long/tidy format.
  - Each variable column becomes a **row**, making it easier for plotting or analysis.
  - Syntax: `pd.melt(id_vars, value_vars, var_name, value_name)`
    1. `id_vars` - columns to keep as identifiers (don't melt).
    2. `value_vars` - columns to unpivot (melt).
    3. `var_name` - new column name for variable names.
    4. `value_name` - new column name for values.

```python
df = pd.DataFrame({
    "country": ["USA", "USA", "India", "India"],
    "year": [2020, 2021, 2020, 2021],
    "sales": [100, 120, 90, 110],
    "profit": [20, 25, 18, 22]
})

melted = df.melt(
    id_vars=["country", "year"],     # columns to keep
    value_vars=["sales", "profit"],  # columns to unpivot
    var_name="metric",               # new column name for variable
    value_name="value"     # new column name for value (default is value)
)

print(melted)
```

- **Pivot (long → wide)**
  - Convert a **long format** DataFrame back into **wide format**.
  - Syntax: `df.pivot(index, columns, values)`
    1. `index` - column to use as row index.
    2. `columns` - column to spread out as new columns.
    3. `values` - column to fill values into the new columns.

```
original = melted.pivot(
    index=["country", "year"],# cols that become the NEW row index
    columns="metric",         # col whose unique values will be the cols
    values="value (in Lakhs)" # col whose values will fill the new df
)

print(pivoted)
```

Combining & Joining Methods

- `df.merge()`

  Merge is used for SQL-like joins & combines two DataFrames based on **common columns (keys)**.

  It supports –

  - Inner join - Only keep common values of both df
  - Outer join - Keep all values of both df & replace missing with NaN
  - Left join - Keep all values of left df & replace missing with NaN
  - Right join - Keep all values of right df & replace missing with NaN

```
# Merging & Joining
df_customers = pd.DataFrame({
    "customer_id": [1, 2, 3, 4],
    "name": ["Adam", "Bob", "Charlie", "Dave"]
})

df_orders = pd.DataFrame({
    "order_id": [101, 102, 103, 104],
    "customer_id": [2, 1, 4, 5],
    "amount": [250, 120, 300, 180]
})

pd.merge(df_customers, df_orders, on="customer_id")              # Inner Join
pd.merge(df_customers, df_orders, on="customer_id", how="left")  # Left Join
pd.merge(df_customers, df_orders, on="customer_id", how="right") # Right Join
pd.merge(df_customers, df_orders, on="customer_id", how="outer") # Outer Join
```

	order_id	customer_id	amount
0	101	2	250
1	102	1	120
2	103	4	300
3	104	5	180

df_orders

	customer_id	name
0	1	Adam
1	2	Bob
2	3	Charlie
3	4	Dave

df_customers

Outer Join

	customer_id	name	order_id	amount
0	1	Adam	102.0	120.0
1	2	Bob	101.0	250.0
2	3	Charlie	Nan	Nan
3	4	Dave	103.0	300.0
4	5	Nan	104.0	180.0

We also have `Join()` method which is essentially a shortcut for merge when using indices. It is used to combine two DataFrames based on index by default & can also join on a key column.

- `df.concat()`

  We use this method for data concatenation i.e. to stack DataFrames **vertically (on top) or horizontally (side by side)**.

```python
df1 = pd.DataFrame({
    "id": [1, 2, 3],
    "name": ["Adam", "Bob", "Charlie"]
})

df2 = pd.DataFrame({
    "id": [4, 5, 6],
    "name": ["David", "Eva", "Frank"]
})

pd.concat([df1, df2])                      # Row wise concatenation
pd.concat([df1, df2], ignore_index=True) # Row wise concatenation - new index
```

	id	name
0	1	Adam
1	2	Bob
2	3	Charlie
3	4	Dave
4	5	Eva
5	6	Frank

```python
pd.concat([df1, df2], axis=1)       # Col wise concatenation
```

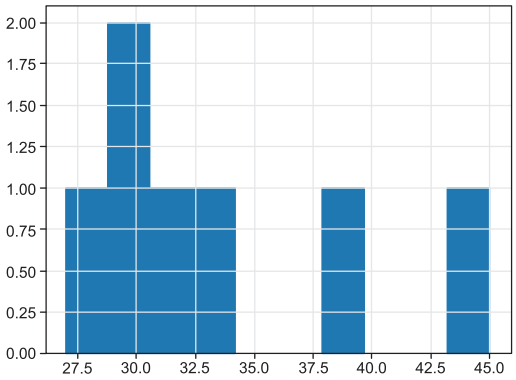
	id	name	id	name
0	1	Adam	4	David
1	2	Bob	5	Eva
2	3	Charlie	6	Frank

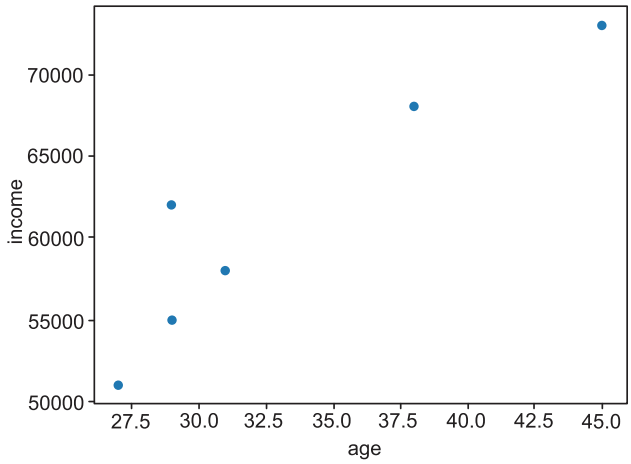
Basic Visualization

Although for most of the data visualization, we are going to use other packages like matplotlib but basic visualization can be done with Pandas using methods like `hist()` & `plot()` .

```python
# Basic Visualization

df["age"].hist()
df.plot(kind='scatter', x='age', y='income')
```

This is what the output looks like:





Keep Learning & Keep Exploring!

Pandas (Assignment Problems)

Let's do some practical exploration & analysis.

Assignment Problems

Q1. Start by loading the **IRIS** dataset. [*Dataset Link:* Iris Flower Dataset]

A. Display the following information:

- First 10 rows
- Shape & Data types
- Summary statistics (mean, std, min, max)

B. Select only those rows where:

- *petal_length > 4.5*
- species = *"Iris-virginica"*

C. Group by *species* and compute:

- Average *sepal_length*
- Maximum *petal_width*
- Standard deviation of *sepal_width*

D. Create a new column: **"petal_ratio" = petal_length / petal_width**

- For each species, find the average petal_ratio.

Q2. Start by loading the **TITANIC** dataset. [*Dataset Link:* Titanic Dataset]

A. Display only columns: *Name, Sex, Age, Fare, Survived*

B. Select passengers who:

- Are female
- Fare > 30

C. Group by *Pclass* and compute:

- Survival rate
- Average Fare
- Average Age

SQL (Structured Query Language)

Structured Query Language

Before learning about SQL, let's first try to understand some basic terms that are a pre-requisite for understanding it.

What is Database?

A **database** is an organized collection of data that is stored and managed electronically so that it can be easily accessed, updated, and analyzed.

Now we could have also stored this data in the form of files (excel, csv etc.) but database provides us with a lot of features/benefits:

- **Organized storage:** Keeps data structured and easy to query.
- **Quick access:** Efficiently retrieve large amounts of data.
- **Data integrity:** Prevents errors and inconsistencies.
- **Multi-user access:** Allows multiple users to work with the data simultaneously.
- **Security:** Control who can access or modify data.
- **Analytics & Reporting:** Essential for dashboards, ML models, and business insights

What is Relational Database?

A **relational database (RDB)** is a type of database that **stores data in structured tables (relations) with rows and columns** and allows relationships between tables using keys.

What is SQL?

SQL (Structured Query Language) is the standard language used to interact with relational databases.

We can use SQL to:

- Retrieve data
- Insert data
- Update data
- Delete data
- Create and manage table

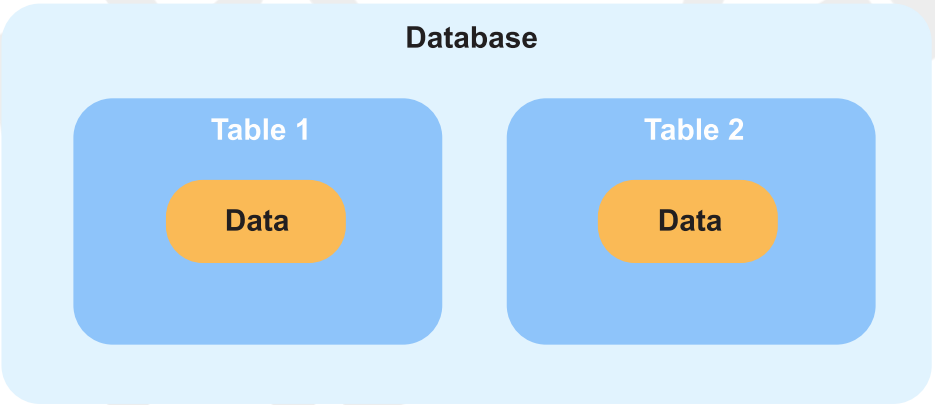
Usage

```sql
SELECT * FROM Customers WHERE Amount > 100;
```

Note - SQL keywords are NOT case sensitive.
Eg - `SELECT` is same as `select` in SQL.

SQL v/s MySQL

SQL is a language used to perform CRUD operations in Relational DB, while MySQL is a RDBMS that uses SQL.



SQL Data Types

In SQL, data types define the kind of data that can be stored in a column or variable.

To See all data types of MYSQL, visit :
https://dev.mysql.com/doc/refman/8.0/en/data-types.html

Here are the frequently used SQL data types:

DATATYPE	DESCRIPTION	USAGE
CHAR	string(0-255), can store characters of fixed length	CHAR(50)
VARCHAR	string(0-255), can store characters up to given length	VARCHAR(50)
BLOB	string(0-65535), can store binary large object	BLOB(1000)
INT	integer(-2,147,483,648 to 2,147,483,647)	INT
TINYINT	integer(-128 to 127)	TINYINT
BIGINT	integer(-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807)	BIGINT
BIT	can store x-bit values. x can range from 1 to 64	BIT(2)
FLOAT	Decimal number - with precision to 23 digits	FLOAT
DOUBLE	Decimal number - with 24 to 53 digits	DOUBLE
BOOLEAN	Boolean values 0 or 1	BOOLEAN
DATE	date in format of YYYY-MM-DD ranging from 1000-01-01 to 9999-12-31	DATE
TIME	HH:MM:SS	TIME
YEAR	year in 4 digits format ranging from 1901 to 2155	YEAR

*Note - CHAR is for fixed length & VARCHAR is for variable length strings. Generally, VARCHAR is better as it only occupies necessary memory & works more efficiently.

We can also use UNSIGNED with datatypes when we only have positive values to add. Eg - UNSIGNED INT

Types of SQL Commands:

1. **DQL** *(Data Query Language)* : Used to retrieve data from databases. (SELECT)

2. **DDL** *(Data Definition Language)* : Used to create, alter, and delete database objects like tables, indexes, etc. (CREATE, DROP, ALTER, RENAME, TRUNCATE)

3. **DML** *(Data Manipulation Language)*: Used to modify the database. (INSERT, UPDATE, DELETE)

4. **DCL** *(Data Control Language)*: Used to grant & revoke permissions. (GRANT, REVOKE)

5. **TCL** *(Transaction Control Language)*: Used to manage transactions. (COMMIT, ROLLBACK, START TRANSACTIONS, SAVEPOINT)

---

1. Data Definition Language (DDL)

Data Definition Language (DDL) is a subset of SQL (Structured Query Language) responsible for defining and managing the structure of databases and their objects.

DDL commands enable you to create, modify, and delete database objects like tables, indexes, constraints, and more.

Key DDL Commands are:

- **CREATE TABLE:**
  - Used to create a new table in the database.
  - Specifies the table name, column names, data types, constraints, and more.
    Example:
    CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(50), salary DECIMAL(10, 2));

- **ALTER TABLE:**
  - Used to modify the structure of an existing table.
  - You can add, modify, or drop columns, constraints, and more.
    Example: ALTER TABLE employees ADD COLUMN email VARCHAR(100);

- **DROP TABLE:**
  - Used to delete an existing table along with its data and structure.
    Example: DROP TABLE employees;

- **CREATE INDEX:**
  - Used to create an index on one or more columns in a table.
  - Improves query performance by enabling faster data retrieval.
    Example: CREATE INDEX idx_employee_name ON employees (name);

- **DROP INDEX:**
  - Used to remove an existing index from a table.
    Example: DROP INDEX idx_employee_name;

- **CREATE CONSTRAINT:**
  - Used to define constraints that ensure data integrity.
  - Constraints include PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK.
    Example: ALTER TABLE orders ADD CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES customers(id);

- **DROP CONSTRAINT:**
  - Used to remove an existing constraint from a table.
    Example: ALTER TABLE orders DROP CONSTRAINT fk_customer;

- **TRUNCATE TABLE:**
  - Used to delete the data inside a table, but not the table itself.
  - Syntax – TRUNCATE TABLE table_name

2. Data Query/Retrieval Language (DQL or DRL)

DQL (Data Query Language) is a subset of SQL focused on retrieving data from databases.

The SELECT statement is the foundation of DQL and allows us to extract specific columns from a table.

- **SELECT:**

  The SELECT statement is used to select data from a database.

  Syntax: SELECT column1, column2, ... FROM table_name;

Here, column1, column2, ... are the field names of the table.

If you want to select all the fields available in the table, use the following syntax:
SELECT * FROM table_name;

Ex: SELECT CustomerName, City FROM Customers;

- **WHERE:**

  The WHERE clause is used to filter records.

  Syntax: SELECT column1, column2, ... FROM table_name WHERE condition;

  Ex: SELECT * FROM Customers WHERE Country='Mexico';

  Operators used in WHERE are:

  = : Equal
  > : Greater than
  < : Less than
  >= : Greater than or equal
  <= : Less than or equal
  <> : Not equal.

  Note: In some versions of SQL this operator may be written as !=

- **AND, OR and NOT:**
  - The WHERE clause can be combined with AND, OR, and NOT operators.
  - The AND and OR operators are used to filter records based on more than one condition:
  - The AND operator displays a record if all the conditions separated by AND are TRUE.
  - The OR operator displays a record if any of the conditions separated by OR is TRUE.
  - The NOT operator displays a record if the condition(s) is NOT TRUE.

  Syntax:

  SELECT column1, column2, ... FROM table_name WHERE condition1 AND condition2 AND condition3 ...;

  SELECT column1, column2, ... FROM table_name WHERE condition1 OR condition2 OR condition3 ...;

SELECT column1, column2, ... FROM table_name WHERE NOT condition;
Example:
SELECT * FROM Customers WHERE Country='India' AND City='Japan';

SELECT * FROM Customers WHERE Country='America' AND (City='India' OR City='Korea');

- **DISTINCT:**

  Removes duplicate rows from query results.

  Syntax: SELECT DISTINCT column1, column2 FROM table_name;

- **LIKE**:

  The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

  There are two wildcards often used in conjunction with the LIKE operator:

  - The percent sign (%) represents zero, one, or multiple characters
  - The underscore sign (_) represents one, single character

  Example: SELECT * FROM employees WHERE first_name LIKE 'J%';

  WHERE CustomerName LIKE 'a%'
  - Finds any values that start with "a"

  WHERE CustomerName LIKE '%a'
  - Finds any values that end with "a"

  WHERE CustomerName LIKE '%or%'
  - Finds any values that have "or" in any position

  WHERE CustomerName LIKE '_r%'
  - Finds any values that have "r" in the second position

  WHERE CustomerName LIKE 'a_%'
  - Finds any values that start with "a" and are at least 2 characters in length

  WHERE CustomerName LIKE 'a__%'
  - Finds any values that start with "a" and are at least 3 characters in length

  WHERE ContactName LIKE 'a%o'
  - Finds any values that start with "a" and ends with "o"

- **IN:**

  Filters results based on a list of values in the WHERE clause.

  Example: SELECT * FROM products WHERE category_id IN (1, 2, 3);

- **BETWEEN**:

  Filters results within a specified range in the WHERE clause.

  Example: SELECT * FROM orders WHERE order_date BETWEEN '2023-01-01' AND '2023-06-30';

- **IS NULL**:

  Checks for NULL values in the WHERE clause.

  Example: SELECT * FROM customers WHERE email IS NULL;

- **AS**:

  Renames columns or expressions in query results.

  Example: SELECT first_name AS "First Name", last_name AS "Last Name" FROM employees;

- **ORDER BY**

  The ORDER BY clause allows you to sort the result set of a query based on one or more columns.

  Basic Syntax:

  - The ORDER BY clause is used after the SELECT statement to sort query results.
  - Syntax: SELECT column1, column2 FROM table_name ORDER BY column1 [ASC|DESC];

  Ascending and Descending Order:

  - By default, the ORDER BY clause sorts in ascending order (smallest to largest).
  - You can explicitly specify descending order using the DESC keyword.
  - Example: SELECT product_name, price FROM products ORDER BY price DESC;

Sorting by Multiple Columns:

- You can sort by multiple columns by listing them sequentially in the ORDER BY clause.
- Rows are first sorted based on the first column, and for rows with equal values, subsequent columns are used for further sorting.
- Example: SELECT first_name, last_name FROM employees ORDER BY last_name, first_name;

Sorting by Expressions:

- It's possible to sort by calculated expressions, not just column values.
- Example: SELECT product_name, price, price * 1.1 AS discounted_price FROM products ORDER BY discounted_price;

Sorting NULL Values:

- By default, NULL values are considered the smallest in ascending order and the largest in descending order.
- You can control the sorting behaviour of NULL values using the NULLS FIRST or NULLS LAST options.
- Example: SELECT column_name FROM table_name ORDER BY column_name NULLS LAST;

Sorting by Position:

- Instead of specifying column names, you can sort by column positions in the ORDER BY clause.
- Example: SELECT product_name, price FROM products ORDER BY 2 DESC, 1 ASC;

- **GROUP BY**

The GROUP BY clause in SQL is used to group rows from a table based on one or more columns.

Syntax:

- The GROUP BY clause follows the SELECT statement and is used to based on specified columns.

- Syntax: SELECT column1, aggregate_function(column2) FROM table_name GROUP BY column1;

- Aggregation Functions:
  - Aggregation functions (e.g., COUNT, SUM, AVG, MAX, MIN) are often used with GROUP BY to calculate values for each group.
  - Example: SELECT department, AVG(salary) FROM employees GROUP BY department;

- Grouping by Multiple Columns:
  - You can group by multiple columns by listing them in the GROUP BY clause.
  - This creates a hierarchical grouping based on the specified columns.
  - Example: SELECT department, gender, AVG(salary) FROM employees GROUP BY department, gender;

- HAVING Clause:
  - The HAVING clause is used with GROUP BY to filter groups based on aggregate function results.
  - It's similar to the WHERE clause but operates on grouped data.
  - Example: SELECT department, AVG(salary) FROM employees GROUP BY department HAVING AVG(salary) > 50000;

- Combining GROUP BY and ORDER BY:
  - You can use both GROUP BY and ORDER BY in the same query to control the order of grouped results.
  - Example: SELECT department, COUNT(*) FROM employees GROUP BY department ORDER BY COUNT(*) DESC;

- **Aggregate Functions**

  These are used to perform calculations on groups of rows or entire result sets. They provide insights into data by summarising and processing information.

  Common Aggregate Functions:

  - COUNT():
    Counts the number of rows in a group or result set.

  - SUM():
    Calculates the sum of numeric values in a group or result set.

  - AVG():
    Computes the average of numeric values in a group or result set.

  - MAX():
    Finds the maximum value in a group or result set.

- MIN():
  Retrieves the minimum value in a group or result set.

3. Data Manipulation Language

Data Manipulation Language (DML) in SQL encompasses commands that manipulate data within a database. DML allows you to insert, update, and delete records, ensuring the accuracy and currency of your data.

- **INSERT:**
  - The INSERT statement adds new records to a table.
  - Syntax: INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
  - Example: INSERT INTO employees (first_name, last_name, salary) VALUES ('John', 'Doe', 50000);

- **UPDATE**:
- The UPDATE statement modifies existing records in a table.
- Syntax: UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;
- Example: UPDATE employees SET salary = 55000 WHERE first_name = 'John';

- **DELETE**:
- The DELETE statement removes records from a table.
- Syntax: DELETE FROM table_name WHERE condition;
- Example: DELETE FROM employees WHERE last_name = 'Doe';

4. Data Control Language (DCL)

Data Control Language focuses on the management of access rights, permissions, and security-related aspects of a database system.

DCL commands are used to control who can access the data, modify the data, or perform administrative tasks within a database.

DCL is an important aspect of database security, ensuring that data remains protected and only authorised users have the necessary privileges.

There are two main DCL commands in SQL: GRANT and REVOKE.

- **GRANT:**

  The GRANT command is used to provide specific privileges or permissions to users or roles. Privileges can include the ability to perform various actions on tables, views, procedures, and other database objects.

  Syntax:

  ```
  GRANT privilege_type
  ON object_name
  TO user_or_role;
  ```

  In this syntax:
  - privilege_type refers to the specific privilege or permission being granted (e.g., SELECT, INSERT, UPDATE, DELETE).
  - object_name is the name of the database object (e.g., table, view) to which the privilege is being granted.
  - user_or_role is the name of the user or role that is being granted the privilege.

  Example: Granting SELECT privilege on a table named "Employees" to a user named "Analyst":

  ```
  GRANT SELECT ON Employees TO Analyst;
  ```

- **REVOKE:**

  The REVOKE command is used to remove or revoke specific privileges or permissions that have been previously granted to users or roles.

  Syntax:

  ```
  REVOKE privilege_type
  ON object_name
  FROM user_or_role;
  ```

  In this syntax:
  - privilege_type is the privilege or permission being revoked.
  - object_name is the name of the database object from which the privilege is being revoked.

- user_or_role is the name of the user or role from which the privilege is being revoked.

Example: Revoking the SELECT privilege on the "Employees" table from the "Analyst" user:

REVOKE SELECT ON Employees FROM Analyst;

DCL and Database Security:

DCL plays a crucial role in ensuring the security and integrity of a database system.

By controlling access and permissions, DCL helps prevent unauthorised users from tampering with or accessing sensitive data. Proper use of GRANT and REVOKE commands ensures that only users who require specific privileges can perform certain actions on database objects.

5. Transaction Control Language (TCL)

Transaction Control Language (TCL) deals with the management of transactions within a database.
TCL commands are used to control the initiation, execution, and termination of transactions, which are sequences of one or more SQL statements that are executed as a single unit of work.
Transactions ensure data consistency, integrity, and reliability in a database by grouping related operations together and either committing or rolling back changes based on the success or failure of those operations.

There are three main TCL commands in SQL: COMMIT, ROLLBACK, and SAVEPOINT.

- **COMMIT:**

  The COMMIT command is used to permanently save the changes made during a transaction.

  It makes all the changes applied to the database since the last COMMIT or ROLLBACK command permanent.
  Once a COMMIT is executed, the transaction is considered successful, and the changes are made permanent.

  Example: Committing changes made during a transaction:

  UPDATE Employees

```
SET Salary = Salary * 1.10
WHERE Department = 'Sales';

COMMIT;
```

- **ROLLBACK:**

  The ROLLBACK command is used to undo changes made during a transaction. It reverts all the changes applied to the database since the transaction began.

  ROLLBACK is typically used when an error occurs during the execution of a transaction, ensuring that the database remains in a consistent state.

  Example: Rolling back changes due to an error during a transaction:

```
BEGIN;

UPDATE Inventory
SET Quantity = Quantity - 10
WHERE ProductID = 101;

-- An error occurs here ROLLBACK;
```

- **SAVEPOINT:**

  The SAVEPOINT command creates a named point within a transaction, allowing you to set a point to which you can later ROLLBACK if needed.

  SAVEPOINTs are useful when you want to undo part of a transaction while preserving other changes.

  Syntax: SAVEPOINT savepoint_name;

  Example: Using SAVEPOINT to create a point within a transaction:

```
BEGIN;

UPDATE Accounts
SET Balance = Balance - 100
WHERE AccountID = 123;

SAVEPOINT before_withdrawal;

UPDATE Accounts
SET Balance = Balance + 100
```

```
WHERE AccountID = 456;
-- An error occurs here
ROLLBACK TO before_withdrawal;
-- The first update is still applied
COMMIT;
```

TCL and Transaction Management:

Transaction Control Language (TCL) commands are vital for managing the integrity and consistency of a database's data.
They allow you to group related changes into transactions, and in the event of errors, either commit those changes or roll them back to maintain data integrity.
TCL commands are used in combination with Data Manipulation Language (DML) and other SQL commands to ensure that the database remains in a reliable state despite unforeseen errors or issues.

Joins

In a DBMS, a join is an operation that combines rows from two or more tables based on a related column between them.
Joins are used to retrieve data from multiple tables by linking them together using a common key or column.

Types of Joins:

1. Inner Join
2. Outer Join
3. Cross Join
4. Self Join

1) Inner Join

An inner join combines data from two or more tables based on a specified condition, known as the join condition.
The result of an inner join includes only the rows where the join condition is met in all participating tables.
It essentially filters out non-matching rows and returns only the rows that have matching values in both tables.

Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Here:

- columns refers to the specific columns you want to retrieve from the tables.
- table1 and table2 are the names of the tables you are joining.
- column is the common column used to match rows between the tables.
- The ON clause specifies the join condition, where you define how the tables are related.

Example: Consider two tables: Customers and Orders.

Customers Table:

CustomerID	CustomerName
1	Alice
2	Bob
3	Carol

Orders Table:

OrderID	CustomerID	Product
101	1	Laptop
102	3	Smartphone
103	2	Headphones

Inner Join Query:

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerName	Product
Alice	Laptop
Bob	Headphones
Carol	Smartphone

2) Outer Join

Outer joins combine data from two or more tables based on a specified condition, just like inner joins. However, unlike inner joins, outer joins also include rows that do not have matching values in both tables.
Outer joins are particularly useful when you want to include data from one table even if there is no corresponding match in the other table.

Types:

There are three types of outer joins: left outer join, right outer join, and full outer join.

1. Left Outer Join (Left Join):

A left outer join returns all the rows from the left table and the matching rows from the right table.

If there is no match in the right table, the result will still include the left table's row with NULL values in the right table's columns.

Example:

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerName	Product
Alice	Laptop
Bob	Headphones
Carol	Smartphone
NULL	Monitor

In this example, the left outer join includes all rows from the Customers table.

Since there is no matching customer for the order with OrderID 103 (Monitor), the result includes a row with NULL values in the CustomerName column.

2. Right Outer Join (Right Join):

A right outer join is similar to a left outer join, but it returns all rows from the right table and the matching rows from the left table.

If there is no match in the left table, the result will still include the right table's row with NULL values in the left table's columns.

Example: Using the same Customers and Orders tables.

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerName	Product
Alice	Laptop
Carol	Smartphone
Bob	Headphones
NULL	Keyboard

Here, the right outer join includes all rows from the Orders table. Since there is no matching order for the customer with CustomerID 4, the result includes a row with NULL values in the CustomerName column.

3. Full Outer Join (Full Join):

A full outer join returns all rows from both the left and right tables, including matches and non-matches.

If there's no match, NULL values appear in columns from the table where there's no corresponding value.

Example: Using the same Customers and Orders tables.

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
FULL OUTER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerName	Product
Alice	Laptop
Bob	Headphones
Carol	Smartphone
NULL	Monitor
NULL	Keyboard

In this full outer join example, all rows from both tables are included in the result. Both non-matching rows from the Customers and Orders tables are represented with NULL values.

3) Cross Join

A cross join, also known as a Cartesian product, is a type of join operation in a Database Management System (DBMS) that combines every row from one table with every row from another table.

Unlike other join types, a cross join does not require a specific condition to match rows between the tables. Instead, it generates a result set that contains all possible combinations of rows from both tables.

Cross joins can lead to a large result set, especially when the participating tables have many rows.

Syntax:

```
SELECT columns
FROM table1
CROSS JOIN table2;
```

In this syntax:

- columns refers to the specific columns you want to retrieve from the cross-joined tables.
- table1 and table2 are the names of the tables you want to combine using a cross join.

Example: Consider two tables: Students and Courses.

Students Table:

StudentID	StudentName
1	Alice
2	Bob

Courses Table:

CourseID	CourseName
101	Maths
102	Science

Cross Join Query:

```
SELECT Students.StudentName, Courses.CourseName
FROM Students
CROSS JOIN Courses;
```

Result:

StudentName	CourseName
Alice	Maths
Alice	Science
Bob	Maths
Bob	Science

In this example, the cross join between the Students and Courses tables generates all possible combinations of rows from both tables. As a result, each student is paired with each course, leading to a total of four rows in the result set.

4) Self Join

A self join involves joining a table with itself.

This technique is useful when a table contains hierarchical or related data and you need to compare or analyse rows within the same table.

Self joins are commonly used to find relationships, hierarchies, or patterns within a single table.

In a self join, you treat the table as if it were two separate tables, referring to them with different aliases.

Syntax:

The syntax for performing a self join in SQL is as follows:

```
SELECT columns
FROM table1 AS alias1
JOIN table1 AS alias2 ON alias1.column = alias2.column;
```

In this syntax:

- columns refers to the specific columns you want to retrieve from the self-joined table.
- table1 is the name of the table you're joining with itself.
- alias1 and alias2 are aliases you assign to the table instances for differentiation.
- column is the column you use as the join condition to link rows from the same table.

Example: Consider an Employees table that contains information about employees and their managers.

Employees Table:

EmployeeID	EmployeeName	ManagerID
1	Alice	3
2	Bob	3
3	Carol	NULL
4	David	1

Self Join Query:

```
SELECT e1.EmployeeName AS Employee, e2.EmployeeName AS Manager
FROM Employees AS e1
JOIN Employees AS e2 ON e1.ManagerID = e2.EmployeeID;
```

Result:

Employee	Manager
Alice	Carol
Bob	Carol
David	Alice

In this example, the self join is performed on the Employees table to find the relationship between employees and their managers. The join condition connects the ManagerID column in the e1 alias (representing employees) with the EmployeeID column in the e2 alias (representing managers).

Set Operations

Set operations in SQL are used to combine or manipulate the result sets of multiple SELECT queries.
They allow you to perform operations similar to those in set theory, such as union, intersection, and difference, on the data retrieved from different tables or queries.

Set operations provide powerful tools for managing and manipulating data, enabling you to analyse and combine information in various ways.

There are four primary set operations in SQL:

- UNION
- INTERSECT
- EXCEPT (or MINUS)
- UNION ALL

1. **UNION:**

The UNION operator combines the result sets of two or more SELECT queries into a single result set.
It removes duplicates by default, meaning that if there are identical rows in the result sets, only one instance of each row will appear in the final result.

Example:

Assume we have two tables: Customers and Suppliers.

Customers Table:

CustomerID	CustomerName
1	Alice
2	Bob

Suppliers Table:

SupplierID	SupplierName
101	SupplierA
102	SupplierB

UNION Query:

```
SELECT CustomerName FROM Customers
UNION
SELECT SupplierName FROM Suppliers;
```

Result:

CustomerName
Alice
Bob
SupplierA
SupplierB

2. **INTERSECT:**

The INTERSECT operator returns the common rows that exist in the result sets of two or more SELECT queries.

It only returns distinct rows that appear in all result sets.

Example: Using the same tables as before.

```
SELECT CustomerName FROM Customers
INTERSECT
SELECT SupplierName FROM Suppliers;
```

Result:

CustomerName

In this example, there are no common names between customers and suppliers, so the result is an empty set.

3. **EXCEPT (or MINUS):**

The EXCEPT operator (also known as MINUS in some databases) returns the distinct rows that are present in the result set of the first SELECT query but not in the result set of the second SELECT query.

Example: Using the same tables as before.

```
SELECT CustomerName FROM Customers
EXCEPT
SELECT SupplierName FROM Suppliers;
```

Result:

CustomerName
Alice
Bob

In this example, the names "Alice" and "Bob" are customers but not suppliers, so they appear in the result set.

4. **UNION ALL:**

The UNION ALL operator performs the same function as the UNION operator but does not remove duplicates from the result set. It simply concatenates all rows from the different result sets.

Example: Using the same tables as before.

```
SELECT CustomerName FROM Customers
```

UNION ALL
SELECT SupplierName FROM Suppliers;

Result:

CustomerName
Alice
Bob
SupplierA
SupplierB

Difference between Set Operations and Joins

Aspect	Set Operations	Joins
Purpose	Manipulate result sets based on set theory principles.	Combine data from related tables based on specified conditions.
Data Source	Result sets of SELECT queries.	Tables that are related by common columns.
Combining Rows	Combine rows from different result sets. May remove duplicates.	Combine rows from different tables based on specified conditions.
Output Columns	Require the SELECT queries to have the same number of output columns and compatible data types.	Can combine columns from different tables, regardless of data types or column numbers.
Common Operations	UNION, INTERSECT, EXCEPT (MINUS).	INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN.
Conditional Requirements	No specific join conditions are required.	Require specified join conditions for combining data.
Handling Duplicates	UNION removes duplicates by default.	Joins do not inherently handle duplicates; it depends on the join type and data.
Usage Scenarios	Useful for combining and analysing related data from different queries or tables.	Used to retrieve and relate data from different tables based on their relationships.
Result Set Structure	Result sets may have different column names, but data types and counts must match.	Result sets can have different column names, data types, and counts.

Performance Considerations	Generally faster and less complex than joins.	Joins can be more complex and resource-intensive, especially for larger datasets.

Sub Queries

Subqueries, also known as nested queries or inner queries, allow you to use the result of one query (the inner query) as the input for another query (the outer query).

Subqueries are often used to retrieve data that will be used for filtering, comparison, or calculation within the context of a larger query.

They are a way to break down complex tasks into smaller, manageable steps.

Syntax:

```
SELECT columns
FROM table
WHERE column OPERATOR (SELECT column FROM table WHERE condition);
```

In this syntax:

- columns refers to the specific columns you want to retrieve from the outer query.
- table is the name of the table you're querying.
- column is the column you're applying the operator to in the outer query.
- OPERATOR is a comparison operator such as =, >, <, IN, NOT IN, etc.
- (SELECT column FROM table WHERE condition) is the subquery that provides the input for the comparison.

Example: Consider two tables: Products and Orders.

Products Table:

ProductID	ProductName	Price
1	Laptop	1000
2	Smartphone	500
3	Headphones	50

Orders Table:

OrderID	ProductID	Quantity
101	1	2
102	3	1

For Example: Retrieve the product names and quantities for orders with a total cost greater than the average price of all products.

```
SELECT ProductName, Quantity
FROM Products
WHERE Price * Quantity > (SELECT AVG(Price) FROM Products);
```

Result:

ProductName	Quantity
Laptop	2

Differences Between Subqueries and Joins:

Aspect	Subqueries	Joins
Purpose	Retrieve data for filtering, comparison, or calculation within the context of a larger query.	Combine data from related tables based on specified conditions.
Data Source	Result of one query used as input for another query.	Data from multiple related tables.
Combining Rows	Not used for combining rows; used to filter or evaluate data.	Combines rows from different tables based on specified join conditions.
Result Set Structure	Subqueries return scalar values, single-column results, or small result sets.	Joins return multi-column result sets.
Performance Considerations	Subqueries can be slower and less efficient, especially when dealing with large datasets.	Joins can be more efficient for combining data from multiple tables.
Complexity	Subqueries can be easier to understand for simple tasks or smaller datasets.	Joins can become complex, but are more suited for handling large-scale data retrieval and combination tasks.
Versatility	Subqueries can be used in various clauses: WHERE, FROM, HAVING, etc.	Joins are primarily used in the FROM clause for combining tables.

Keep Learning & Keep Exploring!

SQL CheatSheet

The Complete SQL Cheat Sheet

1. DDL Commands

Data Definition Language - Defining and optimizing database structure.

Command	Description	Syntax	Example
CREATE	The CREATE command creates a new database and objects, such as a table, index, view, or stored procedure.	`CREATE TABLE table_name (column1 datatype1, column2 datatype2, ...);`	`CREATE TABLE employees (employee_id INT PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50), age INT);`
ALTER	The ALTER command adds, deletes, or modifies columns in an existing table.	`ALTER TABLE table_name ADD column_name datatype;`	`ALTER TABLE customers ADD email VARCHAR(100);`
DROP	The DROP command is used to drop an existing table in a database.	`DROP TABLE table_name;`	`DROP TABLE customers;`
TRUNCATE	The TRUNCATE command is used to delete the data inside a table, but not the table itself.	`TRUNCATE TABLE table_name;`	`TRUNCATE TABLE customers;`

2. DML Commands

Data Manipulation Language Commands - Manipulating data rows.

Command	Description	Syntax	Example
SELECT	The SELECT command retrieves data from a database.	`SELECT column1, column2 FROM table_name;`	`SELECT first_name, last_name FROM customers;`

INSERT	The INSERT command adds new records to a table.	`INSERT INTO table_name (column1, column2) VALUES (value1, value2);`	`INSERT INTO customers (first_name, last_name) VALUES ('Mary', 'Doe');`
UPDATE	The UPDATE command is used to modify existing records in a table.	`UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;`	`UPDATE employees SET employee_name = 'John Doe', department = 'Marketing';`
DELETE	The DELETE command removes records from a table.	`DELETE FROM table_name WHERE condition;`	`DELETE FROM employees WHERE employee_name = 'John Doe';`

3. Querying Data Commands

Command	Description	Syntax	Example
SELECT Statement	The SELECT statement is the primary command used to retrieve data from a database	`SELECT column1, column2 FROM table_name;`	`SELECT first_name, last_name FROM customers;`
WHERE Clause	The WHERE clause is used to filter rows based on a specified condition.	`SELECT * FROM table_name WHERE condition;`	`SELECT * FROM customers WHERE age > 30;`
ORDER BY Clause	The ORDER BY clause is used to sort the result set in ascending or descending order based on a specified column.	`SELECT * FROM table_name ORDER BY column_name ASC|DESC;`	`SELECT * FROM products ORDER BY price DESC;`
GROUP BY Clause	The GROUP BY clause groups rows based on the values in a specified column. It is often used with aggregate functions like COUNT, SUM, AVG, etc.	`SELECT column_name, COUNT(*) FROM table_name GROUP BY column_name;`	`SELECT category, COUNT(*) FROM products GROUP BY category;`
HAVING Clause	The HAVING clause filters grouped results based on a specified condition.	`SELECT column_name, COUNT(*) FROM table_name GROUP BY column_name HAVING condition;`	`SELECT category, COUNT(*) FROM products GROUP BY category HAVING COUNT(*) > 5;`

4. Aggregate Functions Commands

Command	Description	Syntax	Example
COUNT()	The COUNT command counts the number of rows or non-null values in a specified column.	`SELECT COUNT(column_name) FROM table_name;`	`SELECT COUNT(age) FROM employees;`
SUM()	The SUM command is used to calculate the sum of all values in a specified column.	`SELECT SUM(column_name) FROM table_name;`	`SELECT SUM(revenue) FROM sales;`
AVG()	The AVG command is used to calculate the average (mean) of all values in a specified column.	`SELECT AVG(column_name) FROM table_name;`	`SELECT AVG(price) FROM products;`
MIN()	The MIN command returns the minimum (lowest) value in a specified column.	`SELECT MIN(column_name) FROM table_name;`	`SELECT MIN(price) FROM products;`
MAX()	The MAX command returns the maximum (highest) value in a specified column.	`SELECT MAX(column_name) FROM table_name;`	`SELECT MAX(price) FROM products;`

5. Joining Commands

Command	Description	Syntax	Example
INNER JOIN	The INNER JOIN command returns rows with matching values in both tables.	`SELECT * FROM table1 INNER JOIN table2 ON table1.column = table2.column;`	`SELECT * FROM employees INNER JOIN departments ON employees.department_id = departments.id;`
LEFT JOIN/LEFT OUTER JOIN	The LEFT JOIN command returns all rows from the left table (first table) and the matching rows from the right	`SELECT * FROM table1 LEFT JOIN table2 ON table1.column = table2.column;`	`SELECT * FROM employees LEFT JOIN departments ON employees.department_id = departments.id;`

	table (second table).		
RIGHT JOIN/RIGHT OUTER JOIN	The RIGHT JOIN command returns all rows from the right table (second table) and the matching rows from the left table (first table).	`SELECT * FROM table1 RIGHT JOIN table2 ON table1.column = table2.column;`	`SELECT * FROM employees RIGHT JOIN departments ON employees.department_id = departments.department_id;`
FULL JOIN/FULL OUTER JOIN	The FULL JOIN command returns all rows when there is a match in either the left table or the right table.	`SELECT * FROM table1 FULL JOIN table2 ON table1.column = table2.column;`	`SELECT * FROM employees LEFT JOIN departments ON employees.employee_id = departments.employee_id UNION SELECT * FROM employees RIGHT JOIN departments ON employees.employee_id = departments.employee_id;`
CROSS JOIN	The CROSS JOIN command combines every row from the first table with every row from the second table, creating a Cartesian product.	`SELECT * FROM table1 CROSS JOIN table2;`	`SELECT * FROM employees CROSS JOIN departments;`
SELF JOIN	The SELF JOIN command joins a table with itself.	`SELECT * FROM table1 t1, table1 t2 WHERE t1.column = t2.column;`	`SELECT * FROM employees t1, employees t2 WHERE t1.employee_id = t2.employee_id;`
NATURAL JOIN	The NATURAL JOIN command matches columns with the same name in both tables.	`SELECT * FROM table1 NATURAL JOIN table2;`	`SELECT * FROM employees NATURAL JOIN departments;`

6. Subqueries in SQL

Command	Description	Syntax	Example
IN	The IN command is used to determine whether a	`SELECT column(s) FROM table WHERE`	`SELECT * FROM customers WHERE city IN`

	value matches any value in a subquery result. It is often used in the WHERE clause.	`value IN (subquery);`	`(SELECT city FROM suppliers);`
ANY	The ANY command is used to compare a value to any value returned by a subquery. It can be used with comparison operators like =, >, <, etc.	`SELECT column(s) FROM table WHERE value < ANY (subquery);`	`SELECT * FROM products WHERE price < ANY (SELECT unit_price FROM supplier_products);`
ALL	The ALL command is used to compare a value to all values returned by a subquery. It can be used with comparison operators like =, >, <, etc.	`SELECT column(s) FROM table WHERE value > ALL (subquery);`	`SELECT * FROM orders WHERE order_amount > ALL (SELECT total_amount FROM previous_orders);`

> *Keep Learning & Keep Exploring!*

SQL (Assignment)

Employee Table

EmpID	FirstName	LastName	Department	Salary	HireDate
101	Alice	Johnson	IT	6500	2020-03-15
102	Mark	Rivera	HR	4800	2019-07-22
103	Sophia	Lee	Finance	7200	2021-01-10
104	Daniel	Kim	IT	5800	2018-11-05
105	Emma	Brown	Marketing	5300	2022-04-18
106	Liam	Patel	Finance	6900	2020-09-29
107	Olivia	Garcia	HR	4600	2017-06-30
108	Noah	Thompson	IT	7500	2023-02-12
109	Ava	Martinez	Marketing	5100	2019-12-02
110	Ethan	Davis	Finance	8000	2016-05-14

Create the above Table in MySQL and solve the following problems based on it.

Assignment Problems

Q1. Write a query to display every employee and all their data.

Q2. List only the FirstName, LastName, and Salary of every employee.

Q3. Show all employees who work in the **'IT'** department.

Q4. Retrieve employees with a salary **greater than 6000**.

Q5. List all employees ordered by **HireDate from newest to oldest**.

Q6. Show a list of all **unique departments** present in the table.

Q7. Find employees whose first name **starts with 'A'**.

Q8. Show employees whose salaries are **between 4000 and 7000**.

Q9. Find the **average salary** of all employees.

Q10. List each department along with the **number of employees**, but only include departments with **more than 3 employees**.

Transactions

ACID Properties

- **Atomicity** - All statements succeed or none succeed.
- **Consistency** - Data moves from one valid state to another.
- **Isolation** - Parallel transactions don't interfere.
- **Durability** - Committed data is permanently saved.

Transactions

Disable autocommit

```
SET autocommit = 0;
```

Enable autocommit

```
SET autocommit = 1;
```

Transactions

Start & Commit

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 50 WHERE id = 1;
UPDATE accounts SET balance = balance + 50 WHERE id = 2;

COMMIT;
```

Transactions

Rollback

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 3;

ROLLBACK;
```

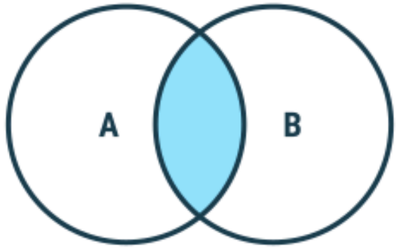
Transactions

Savepoint

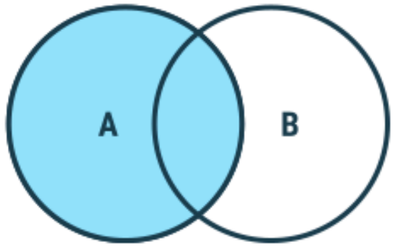
```
START TRANSACTION;

UPDATE accounts SET balance = balance + 1000 WHERE id = 1;
SAVEPOINT after_wallet_topup;

UPDATE accounts SET balance = balance + 10 WHERE id = 1;

ROLLBACK TO after_wallet_topup;

COMMIT;
```

JOINs

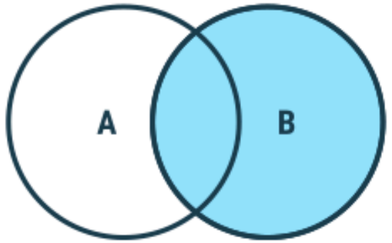
JOINs are used to combine rows from two or more tables based on a related column between them.



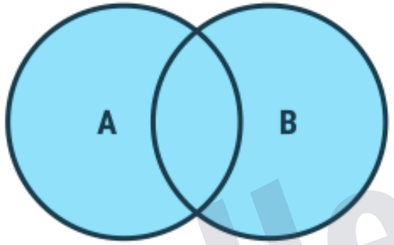
Inner Join



Left Join



Right Join

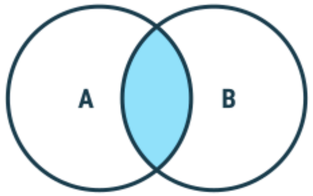


Full Join

Outer Joins

JOINs

INNER JOIN



customer_id	name	city
1	Alice	Mumbai
2	Bob	Delhi
3	Charlie	Bangalore
4	David	Mumbai

customers

order_id	customer_id	amount
101	1	500
102	1	900
103	2	300
104	5	700

orders

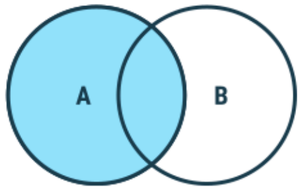
Syntax

SELECT *column(s)*
FROM ***tableA***
INNER JOIN ***tableB***
ON *tableA.col_name = tableB.col_name;*

```sql
-- inner join
SELECT c.name, o.order_id, o.amount
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

JOINs

LEFT JOIN



customer_id	name	city
1	Alice	Mumbai
2	Bob	Delhi
3	Charlie	Bangalore
4	David	Mumbai

customers

order_id	customer_id	amount
101	1	500
102	1	900
103	2	300
104	5	700

orders

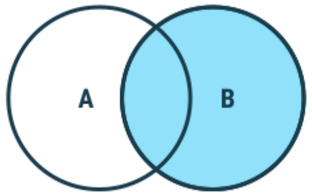
Syntax

SELECT *column(s)*
FROM ***tableA***
LEFT JOIN ***tableB***
ON *tableA.col_name = tableB.col_name;*

```sql
-- left join
SELECT *
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

JOINs

RIGHT JOIN



customer_id	name	city
1	Alice	Mumbai
2	Bob	Delhi
3	Charlie	Bangalore
4	David	Mumbai

customers

order_id	customer_id	amount
101	1	500
102	1	900
103	2	300
104	5	700

orders

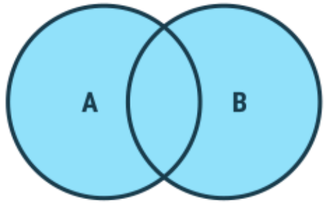
Syntax

SELECT *column(s)*
FROM ***tableA***
RIGHT JOIN ***tableB***
ON *tableA.col_name = tableB.col_name;*

```sql
-- right join
SELECT *
FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;
```

JOINs

OUTER JOIN



LEFT JOIN

UNION

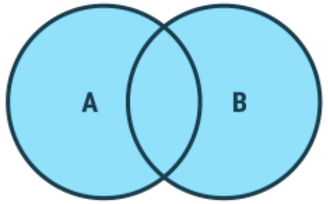
RIGHT JOIN

Syntax in MySQL

```
SELECT * FROM customers as c
LEFT JOIN orders as o
ON c.customer_id = o.customer_id
UNION
SELECT * FROM customers as c
RIGHT JOIN orders as o
ON c.customer_id = o.customer_id;
```

JOINs

CROSS JOIN

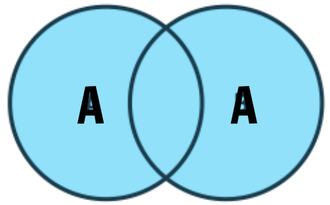


Syntax

SELECT *column(s)*
FROM ***tableA***
CROSS JOIN ***tableB*** ;

```sql
-- cross join
SELECT *
FROM customers as c
CROSS JOIN orders as o;

-- inner join
SELECT *
FROM customers as A
JOIN customers as B
ON A.customer_id = B.customer_id;
```

JOINs



SELF JOIN

It is a regular join but the table is joined with itself.

Syntax

```
SELECT column(s)
FROM table as a
JOIN table as b
ON a.col_name = b.col_name;
```

Practice Qs

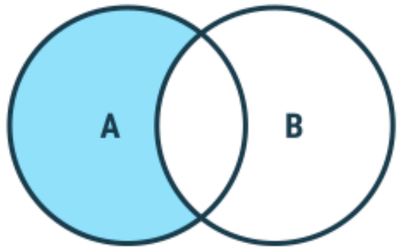
Write SQL command to display the exclusive joins :

customer_id	name	city
1	Alice	Mumbai
2	Bob	Delhi
3	Charlie	Bangalore
4	David	Mumbai

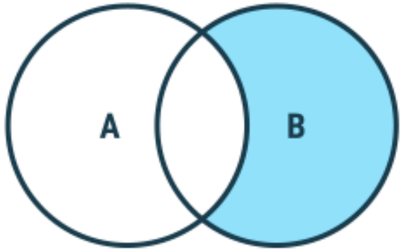
customers

order_id	customer_id	amount
101	1	500
102	1	900
103	2	300
104	5	700

orders



Left Exclusive Join

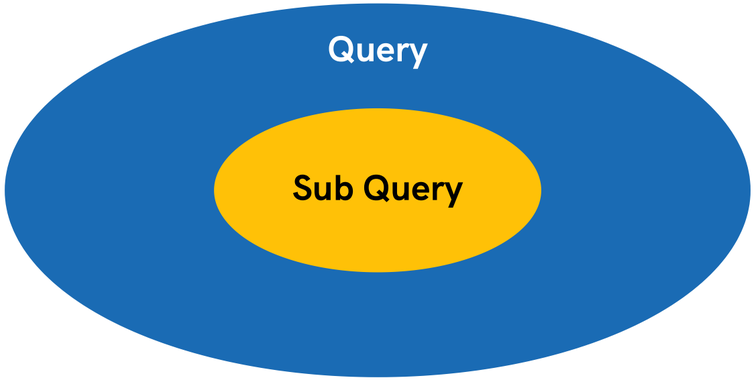


Right Exclusive Join

```sql
-- left exclusive
SELECT *
FROM customers as c
LEFT JOIN orders as o
ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

```sql
-- right exclusive
SELECT *
FROM customers as c
RIGHT JOIN orders as o
ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

Sub-Queries



A Subquery or Inner query or a Nested query is a query within another SQL query. It involves 2 select statements.

Syntax

SELECT *column(s)*
FROM *table_name*
WHERE *col_name operator*
(subquery);

Sub-Queries

With WHERE

```sql
SELECT *
FROM orders
WHERE amount > (
    SELECT AVG(amount)
    FROM orders
);
```

Sub-Queries

With SELECT

```sql
SELECT name,
    (   SELECT COUNT(*)
        FROM orders o
        WHERE o.customer_id = c.customer_id
    ) as order_count
FROM customers c;
```

Sub-Queries

With FROM

```sql
SELECT
    summary.customer_id,
    summary.avg_amount
FROM
    (
        SELECT
            customer_id,
            AVG(amount) AS avg_amount
        FROM orders
        GROUP BY customer_id
    ) AS summary;
```

Views in SQL

A view is a virtual table based on the result-set of an SQL statement.

Syntax

CREATE VIEW *view1 AS*
SELECT *col1, col2* **FROM** *table_name;*

*A view always shows up-to-date data.
The database engine recreates the view,
every time a user queries it.

Views in SQL

- No data is stored physically (unless it's a materialized view in some DBs).
- Can include columns from one or more tables.
- Can be used in SELECT, JOIN, or even WHERE clauses like a normal table.
- Helps with security by exposing only certain columns to users.

Index in SQL

indexes are special database objects that make data retrieval faster.

Syntax (single col & multi-col)

CREATE INDEX *idx_name* ON *table(col);*

CREATE INDEX *idx_name* ON *table(col1, col2);*

SHOW INDEX FROM *table;*

DROP INDEX *idx_name* ON *table;*

Stored Procedures

Predefined set of SQL statements that you can save in the database and execute whenever needed.

Syntax (Create)

```
CREATE PROCEDURE procedure_name (parameters)
BEGIN
    -- SQL statements
END;
```



```
DELIMITER $$

CREATE PROCEDURE check_balance(IN acc_id INT, OUT bal DECIMAL(10, 2))
BEGIN
    SELECT balance INTO bal
    FROM bank_accounts as b
    WHERE b.account_id = acc_id;
END $$

DELIMITER ;

CALL check_balance(2, @balance);
SELECT @balance;
```

Stored Procedures

Syntax (Call)

CALL *procedure_name (arguments);*

Syntax (Drop)

DROP PROCEDURE **IF EXISTS** *procedure_name;*

Data Visualization - Matplotlib

Data Visualization

We, as humans, process visual information faster than text or numbers, so we use **Data visualization** i.e. the graphical representation of data to:

- Understand patterns and trends
- Identify outliers and anomalies
- Communicate insights clearly
- Support data-driven decision making

The common types of plots used in data visualization are line plots, bar charts, histograms, scatter plots, box plots etc.

Matplotlib

Matplotlib is a **Python library** used for:

- Creating static, animated, and interactive plots
- Low-level control over plot appearance
- Serving as the foundation for other libraries (Seaborn, Pandas plotting)

Installation

```
pip install matplotlib
```

Usage

```
import matplotlib.pyplot as plt
```

Basic Plot Structure

```
plt.plot(x, y)
plt.xlabel("X-axis label")
plt.ylabel("Y-axis label")
plt.title("Title")
plt.show()
```

Important Plots in Matplotlib

Line Plots

A line plot is a type of chart used to display data points connected by straight lines, typically to show trends or changes over a continuous variable, such as time or distance.

Example:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x, y)
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Simple Line Plot")
plt.grid(True) # Adds a grid
plt.show()
```

Key features of a line plot:

- **X-axis**: usually continuous (time, sequence, measurements)
- **Y-axis**: values being measured
- **Line style**: solid (), dashed (`-`), dotted (`:`) etc.
- **Markers**: shows individual data points (`o` , `s` , `^` , `'*'` , `'.'`)

Styling a Line Plot

```
plt.plot(x, y,
         color="blue",
         linestyle="--",
         linewidth=2,
         marker="o",
         markersize=6)

plt.show()
```

Multiple Lines on the Same Plot

```
y2 = [1, 3, 5, 7, 9]

plt.plot(x, y, label="Line 1")
plt.plot(x, y2, label="Line 2")

plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```

Saving Plots

```
plt.savefig("line_plot.png")
```

When to use?

Use a line plot when:

- Data points are ordered
- You want to show progression or trends
- You're comparing changes rather than individual categories

In Matplotlib, `fmt` strings are a compact way to specify the line style, marker, and color in a single string, so `fmt` is a combination of `[color][marker][linestyle]`.

Example:

```
plt.plot(x, y, 'g--') # green dashed line
plt.plot(x, y, 'bo')  # blue circles, no line
```

Refer to documentation for all options - https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

Bar Charts

A bar chart is a graph that displays categorical data using rectangular bars. Each bar's height (or length, in horizontal bars) represents a value.

Example:

```
import matplotlib.pyplot as plt

x = ['A', 'B', 'C', 'D']
y = [10, 15, 7, 12]

plt.bar(x, y)
plt.xlabel("Categories")
plt.ylabel("Values")
plt.title("Simple Bar Chart")
plt.show()
```

This creates a vertical bar chart (most common).

Styling a Bar Chart

```
plt.bar(x, y,
        color='skyblue',
        width=0.6,
        edgecolor='black',
        linewidth=1.5)

plt.text()
plt.show()
```

Horizontal Bar Chart

```
plt.barh(x, y)
plt.xlabel("Values")
plt.ylabel("Categories")
plt.title("Horizontal Bar Chart")
plt.show()
```

Multiple Bar Charts (Grouped Bars)

```
import numpy as np

x = np.arange(4)
y1 = [10, 15, 7, 12]
y2 = [8, 14, 9, 10]

width = 0.35

plt.bar(x - width/2, y1, width, label='Group 1')
plt.bar(x + width/2, y2, width, label='Group 2')

plt.xticks(x, ['A', 'B', 'C', 'D'])
plt.xlabel("Categories")
plt.ylabel("Values")
plt.legend()
plt.show()
```

When to use?

Use a bar chart when:

- You are comparing **different categories**
- The data is **discrete**, not continuous
- You want clear visual comparison

We can use `plt.text()` to add text annotations to the bars in the bar chart.

Syntax:

```
values = [10, 20, 30]
plt.bar(['A', 'B', 'C'], values)

for i, v in enumerate(values):
plt.text(i, v, str(v))
```

Scatter Plots

A scatter plot shows the relationship between two variables by plotting points on a 2D plane.

Example:

```
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 3, 5]

plt.scatter(x, y)
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Simple Scatter Plot")
plt.grid(True)
plt.show()
```

Styling a Bar Chart

```
plt.scatter(x, y,
            s=100,            # marker size
            c='red',          # color
            marker='o',
            alpha= 0.7)       # transparency
plt.show()
```

Using a Colormap (Color by Value)

```
import numpy as np

colors = np.array([10, 20, 30, 40, 50])

plt.scatter(x, y, c=colors, cmap='viridis')
plt.colorbar(label='Color Scale')
plt.show()
```

Multiple Scatter Plots

```
y2 = [5, 3, 4, 2, 1]

plt.scatter(x, y, label='Set 1')
plt.scatter(x, y2, label='Set 2')

plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```

When to use?

Use a scatter plot when:

- You want to analyze the **relationship** between two variables
- Data is **numerical and continuous**
- You don't want to imply a trend by connecting points

We can also add annotations. **Annotations** are used to add explanatory text or markers to specific points in a plot.

Example:

```
x = [1, 2, 3, 4, 5]
y = [5, 7, 6, 8, 7]

plt.scatter(x, y)

# Annotate each point
for i in range(len(x)):
    plt.text(x[i]+0.1, y[i]+0.1, f"({x[i]}, {y[i]})") # small offset
```

Pie Charts

A pie chart is a circular chart divided into slices to show relative proportions of a whole.

- Each slice = a category
- Size of slice = value of that category relative to the total

We prefer them only when our data has **few categories**.

Example:

```
sizes = [30, 20, 25, 25]
labels = ['A', 'B', 'C', 'D']

plt.pie(sizes, labels=labels)
plt.show()
```

Adding Percentages

```
plt.pie(sizes,
        labels=labels,
        autopct='%1.1f%%') # shows percentage

plt.show()
```

Changing Colors

```
colors = ['skyblue', 'lightgreen', 'pink', 'orange']

plt.pie(sizes, labels=labels, autopct='%1.1f%%', colors=colors)
plt.show()
```

Starting Angle and Shadow

```
plt.pie(sizes,
        labels=labels,
        autopct='%1.1f%%',
        startangle=90,  # rotate chart
        shadow=True)    # adds shadow

plt.show()
```

Wedgeprops

In Matplotlib pie charts, `wedgeprops` is a dictionary of properties that controls the appearance of the pie slices (wedges). It has common properties like edgecolor, linewidth, linestyle, alpha etc.

```
plt.pie(sizes,
        labels=labels,
        autopct='%1.1f%%',
        wedgeprops={'edgecolor': 'black', 'linewidth': 2})
```

When to use?

Use a pie chart when:

- You want to display **part-to-whole relationships**
- You have **categorical data**
- Data has **few categories** (4–6); too many slices make it hard to read

Histograms

A histogram shows the distribution of numerical data by:

- Dividing values into **bins** (ranges)
- Counting how many values fall into each bin (frequency)

Example:

```
data = [1, 2, 2, 3, 3, 3, 4, 4, 5, 5, 5, 5]

plt.hist(data, bins=5, color='skyblue', edgecolor='black')

plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Simple Histogram")
plt.grid(axis='y', linestyle='--', alpha=0. 7)

plt.show()
```

Number of bins

`bins` controls how the data is divided. It can be an integer or a sequence of bin edges:

```
plt.hist(data, bins=3) # integer

plt.hist(data, bins=[1, 2, 3, 4, 5, 6]) # sequence
```

Histogram Orientation

- Vertical (default)

```
plt.hist(data)
```

- Horizontal

```
plt.hist(data, orientation='horizontal')
```

Multiple Histograms

```
data2 = [2, 3, 3, 4, 4, 5, 5, 5, 6]

plt.hist(data, bins=5, alpha=0.5, label='Data 1', color='blue')
plt.hist(data2, bins=5, alpha=0.5, label='Data 2', color='red')
plt.legend()
```

`alpha` controls transparency so overlapping histograms are visible.

When to use?

Use a histogram when:

- You want to show the **distribution of continuous data**.
- You have to analyze **frequency, spread, skewness**, or patterns.
- Working with **large datasets** to summarize trends.

`axvline` (Axis Vertical Line) is used to draw a **vertical line** at a specific x-value in a plot.

Example:

```
plt.axvline(x=value, color='color', linestyle='style', linewidth=width, label='label')
```

Box Plots

A box plot is a statistical visualization that summarizes the distribution of a dataset using five key numbers:

- Minimum (lowest non-outlier)
- First Quartile (Q1) – 25th percentile
- Median (Q2) – 50th percentile
- Third Quartile (Q3) – 75th percentile
- Maximum (highest non-outlier)

It can also show **outliers**.

Example:

```
data = [7, 8, 5, 6, 9, 7, 8, 10, 4, 6]

plt.boxplot(data)
plt.ylabel("Values")
plt.title("Simple Box Plot")
plt.show()
```

Main Values in the Box Plot

1. **Minimum (Lower Whisker)**
   - The smallest data point within 1.5 × IQR below Q1
   - Any smaller points are considered outliers

2. **First Quartile (Q1)**
   - The 25th percentile of the data
   - 25% of data points are below Q1
   - Bottom edge of the box

3. **Median (Q2)**
   - The 50th percentile (middle value)
   - Line inside the box
   - Splits the dataset into two halves

4. **Third Quartile (Q3)**
   - The 75th percentile of the data
   - 75% of data points are below Q3
   - Top edge of the box

5. **Maximum (Upper Whisker)**
   - The largest data point within 1.5 × IQR above Q3
   - Points beyond this are outliers

6. **Interquartile Range (IQR)**
   - Difference between Q3 and Q1: $\text{IQR} = \text{Q3} - \text{Q1}$
   - Represents the middle 50% of the data

7. **Outliers**
   - Data points outside 1.5 × IQR from Q1 or Q3
   - Plotted as dots or asterisks beyond the whiskers

Horizontal Box Plot

```
plt.boxplot(data, vert=False)
plt.xlabel("Values")
plt.show()
```

Multiple Box Plots

```
data1 = [7, 8, 5, 6, 9, 7, 8, 10, 4, 6]
data2 = [5, 6, 7, 8, 5, 4, 6, 7, 5, 6]

plt.boxplot([data1, data2], labels=['Dataset 1', 'Dataset 2'])
plt.show()
```

Each box represents a different dataset.

Showing Mean

```
plt.boxplot(data, showmeans=True, meanline=True)
```

- `showmeans=True` adds the mean marker.
- `meanline=True` draws a line instead of a point.

When to use?

- Visualize spread and skewness of data
- Identify outliers
- Compare distributions across multiple groups

Stack Plots

A stack plot is a type of plot where multiple data series are stacked on top of each other. Each “layer” shows the contribution of one category, and the top line shows the cumulative total.

Example:

```
x = [1, 2, 3, 4, 5]
y1 = [1, 2, 3, 4, 5]
y2 = [2, 1, 2, 1, 2]

plt.stackplot(x, y1, y2, labels=['Data1', 'Data2'], colors=['blue', 'green'])

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Basic Stack Plot")
plt.legend(loc='upper left')

plt.show()
```

When to use?

Use a stack plot when you want to:

- Show cumulative data over time
- Compare multiple components
- Highlight trends in parts and whole

Subplots

In Matplotlib, subplots allow you to display multiple plots in a single figure, arranged in a grid. This is useful for comparing different datasets or visualizations side by side.

Example:

```
x = [1, 2, 3, 4, 5]
y1 = [2, 3, 5, 7, 11]
y2 = [1, 4, 2, 5, 3]

# 1 row, 2 columns, first subplot
plt.subplot(1, 2, 1) # (rows, columns, index)
plt.plot(x, y1, color='blue', marker='o')
plt.title("First Subplot")
plt.xlabel ("X")
plt.ylabel("Y1")

# 1 row, 2 columns, second subplot
plt.subplot(1, 2, 2)
plt.plot(x, y2, color='red', marker='s')
plt.title("Second Subplot")
plt.xlabel("X")
plt.ylabel("Y2")

plt.tight_layout()
plt.show()
```

- `plt.subplot(nrows, ncols, index)` selects the current axes.
- Index counts row-wise from top-left to bottom-right (Row major).

Modern Matplotlib

Modern Matplotlib with the **Object-Oriented (OO)** approach is now the recommended way to create professional plots.

Why Use OO Style?

- More control over multiple axes, subplots, and figures
- Cleaner and more readable for complex plots
- Avoids side effects of `plt` (state-based interface)
- Easier to combine multiple plot types

Basic Plot

```
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# Create figure and axes
fig, ax= plt.subplots()

# Plot data
ax.plot(x, y, label="Prime numbers", color='blue', linestyle='--', marker='o')

# Add labels and title
ax.set_xlabel("X axis")
ax.set_ylabel("Y axis")
ax.set_title("OO Line Plot Example")

# Add legend and grid
ax.legend()
ax.grid(True)

plt.show()
```

`fig` and `ax` are core objects that give you full control over our plots.

- `fig` is Figure & it represents the entire figure or canvas.
- `ax` is Axes & it represents a single plot or graph within the figure.

Subplots

```
fig, axs = plt.subplots(2, 2, figsize=(8, 6))

axs[0, 0].plot(x, y1)
axs[0, 0].set_title("Top Left")

axs[0, 1].bar(['A','B','C'], [3,5,2])
axs[0, 1].set_title("Top Right")

axs[1, 0].scatter(x, y2)
axs[1, 0].set_title("Bottom Left")

axs[1, 1].hist([1,2,2,3,3,3,4])
axs[1, 1].set_title("Bottom Right")

plt.tight_layout()
plt.show()
```

> *Keep Learning & Keep Exploring!*

Data Visualization – Seaborn

Seaborn

Seaborn is a high-level Python data visualization library **built on top of Matplotlib**. It provides a simpler and more attractive way to create statistical graphics, with default themes and color palettes that look modern and clean.

Key Features

1. **Integration with Pandas** – Works seamlessly with DataFrames and handles column names directly.
2. **Complex plots made easy** – Automatically handles things like regression lines, aggregation, and confidence intervals.
3. **Statistical plotting** – Supports visualization of distributions, relationships, and categorical data (e.g., `boxplot`, `violinplot`, `heatmap`).
4. Beautiful default styles & in-built datasets for learning.

Usage

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load example dataset
tips = sns.load_dataset("tips")

# Scatter plot of total bill vs tip
sns.scatterplot(data=tips, x="total_bill", y="tip", hue="day", style="sex", size="size")
plt.title("Tips Scatter Plot")
plt.show()
```

- `hue` - color by category
- `style` - marker style by category
- `size` - size of points by a variable

Why Use Seaborn

- Saves time with prettier defaults than Matplotlib.
- Great for exploratory data analysis (EDA).
- Works well with complex datasets and statistical visualizations.

Common Plots with Seaborn

Relational Plots

Relational plots are plots that show the relationship between two (or more) variables, typically numerical x and y variables.

Line Plots

```
# Example dataset
data = sns.load_dataset("tips")

# Line plot: average tip by total bill
sns.lineplot(data=data, x="total_bill", y="tip")
plt.title ("Line Plot: Total Bill vs Tip")
plt.show()
```

We can also use `ci` i.e. Confidence Interval.

Scatter Plots

```
# Scatter plot
sns.scatterplot(data=data, x="total_bill", y="tip")
plt.title("Scatter Plot: Total Bill vs Tip")
plt.show()
```

Categorical Plots

Categorical plots are used to visualize data where one variable is categorical (discrete) and the other is usually numerical.

Bar Plots

```
tips = sns.load_dataset("tips")
sns.barplot(data=tips, x="day", y="total_bill", hue="sex")
plt.title("Average Total Bill per Day by Sex")
plt.show()
```

Box Plots

```
sns.boxplot(data=tips, x="day", y="total_bill", hue="sex")
plt.title("Box Plot of Total Bill by Day and Sex")
plt.show()
```

Distribution Plots

Distribution plots are used to visualize the distribution of a single numerical variable.

Histograms

```
tips = sns.load_dataset("tips")
sns.histplot(data=tips, x="total_bill", bins=10, kde=False)
plt.title("Histogram of Total Bill")
plt.show()
```

Matrix Plots

Matrix plots are used to visualize 2D data arranged in a matrix or table, such as correlation matrices, confusion matrices, or pivot tables.

Heatmap

```
# Example: correlation matrix of tips dataset
tips = sns.load_dataset("tips")
corr = tips.corr() # correlation between numerical columns

sns.heatmap (corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

Best Practices

1. When working with data visualization we use Seaborn for **High-Level Plots** and Matplotlib for **Low-Level Customizations**.
2. We work with `fig` and `ax` objects for modular and reusable code.

Creating Simple Plot

```
import matplotlib.pyplot as plt
import seaborn as sns

fig, ax = plt.subplots(figsize=(7,5))

sns.scatterplot(data=sns.load_dataset("tips"), x="total_ bill", y="tip", hue="day", ax=ax)
sns.set_style("whitegrid") # options: white, dark, whitegrid, ticks

ax.set_title("Scatter Plot with Seaborn and Matplotlib")
plt.show()
```

Using Subplots

```
fig, ax = plt.subplots()

sns.boxplot(data=sns.load_dataset("tips"), x="day", y="total_bill", hue="sex", ax=ax)

ax.set_title("Boxplot: Total Bill by Day and Sex")
ax.set_xlabel("Day of the Week")
ax.set_ylabel("Total Bill ($)")
plt.legend(title="Sex")

plt.show()
```

Keep Learning & Keep Exploring!

Data Visualisation (Assignment)

Assignment Problems

Q1. For the following dataset generated using NumPy:

```
np.random.normal(loc=70, scale=10, size=200)
```

- Plot a **histogram** of scores
- Draw a vertical line for the mean & try to interpret the results.

Q2. For the Seaborn `penguins` dataset, do the following:

- Plot `flipper_length_mm` vs `body_mass_g`
- Color by `species`

Q3. For the Seaborn `tips` dataset, do the following:

- Create a **box plot** of `total_bill` for each `day`
- Add `hue="sex"`
- Identify which day has the highest median bill
- Comment on outliers

Q4. For the Seaborn `tips` dataset, do the following:

- Create a **scatter plot** of `total_bill` vs `tip`
- Color points by `sex`
- Use different markers for `smoker`
- Add a title and axis labels
- Analyze: *Do smokers tend to tip differently?*

Q5. For the Seaborn `tips` dataset, do the following:

- Create a pivot table of **average tip**
  - Rows: `day`
  - Columns: `time`
- Plot the pivot table as a **heatmap**
- Annotate values
- Interpret which day–time combination performs best